In [424…

```python
# Import des utilitaires

## librairies de traitement de données
import pandas as pd
import numpy as np

## librairies de gestion de fichiers
import os

## librairies de visualisation des valeurs manquantes
import missingno as msno

## librairies de statistiques et de machine learning
import scipy.stats as st
from sklearn.linear_model import LinearRegression

## librairies de visualisation
import matplotlib.pyplot as plt
import seaborn as sns
```

In [425…

```python
# Fonctions pour importer tout type de fichier en determinant le type de fichier et son delimiter

##Fonction pour récupérer le separateur
def separateurDetect(fichier):

    with open(fichier, mode= "r") as f: #encoding= encodage
        premieres_lignes = f.readline(15)
    for sep in [',', ';', "\t", '|', '|']:
        if sep in premieres_lignes:
            return sep
    return ',' # Par défaut

#Fonction pour importer un fichier
def importFichier(fichier):

    # Le fichier existe ?
    if not os.path.exists(fichier):
        raise FileExistsError(f"Le fichier {fichier} est introuvable.")

    # Détecton le type du fichier
    extension = os.path.splitext(fichier)[1].lower()

    if extension in ['.csv', '.tsv', '.txt']:
        # Détection du séparateur
        separateur = separateurDetect(fichier)
        print(f"Chargement d'un fichier avec pour séparateur '{separateur}'.")
        return pd.read_csv(fichier, sep= separateur)

    elif extension in ['.xls', '.xslx']:
        print(f"Chargement d'un fichier Excel .")#
        return pd.read_excel(fichier, engine= 'openpyxl')

    else:
        raise ValueError(f"Format de fichier non pris en compte : {extension}")
```

In [426…

```python
# Fonctions pour analyser les données

## Fonction pour catégoriser les variables
def categoriseVariables(df):

    info_type = df.dtypes
    variable_qualitative = []
    variable_quantitative = []

    for index in info_type.index:
        if (info_type[index] == 'float') | (info_type[index] == 'int'):
            variable_quantitative.append(index)
        else:
            variable_qualitative.append(index)
    return variable_qualitative, variable_quantitative

## Fonction pour décrire une dataframe
def infoData(df):
    info = df.info().pd.DataFrame()
    return info

def descriptionData(df):
    nb_lignes, nb_col = df.shape
    print(f"La dataframe contient {nb_lignes} lignes et {nb_col} colonnes.")

## Fonction pour afficher les afficher les colonnes ayant un taux de remplissage inférieur à un seuil
def colManquantes(df, seuil):
    # Calculer le taux de remplissage
    taux_remplissage = df.notnull().mean() * 100
    # Sélection des colonnes avec un taux de remplissage < seuil
    colonnes = taux_remplissage[taux_remplissage < seuil].index.tolist()
    print(f"nombre de colonnes avec un taux de remplissage inférieur à {seuil}% : {len(colonnes)}")
    return colonnes

## Fonction pour supprimer des colonnes dans une dataframe
def supprimeCol(df, colonnes):
    return df.drop(columns = colonnes)

## Fonction pour selectionner des colonnes dans une dataframe
def colSelection(df, colonnes):
    return df[colonnes]
```

In [427…

```python
# Fonctions pour remplacer les valeurs manquantes

## Remplacer les valeurs manquantes en utilisant le coeficient d'asymétrie pour choisir la méthode de remplacement
def remplacerValParStats(dataframe, colonnes):

    for colonne in colonnes:
        # Calculer le coefficient d'asymétrie
        skewness = st.skew(dataframe[colonne].dropna())
        #print(f"Coefficient d'asymétrie pour {colonne}: {skewness}")

        # Déterminer la méthode statistique appropriée
        if abs(skewness) < 0.5:
            # Distribution symétrique
            methode = "moyenne"
            valeur_remplacement = dataframe[colonne].mean()
        else:
            # Asymétrie positive/négative
            methode = "médiane"
            valeur_remplacement = dataframe[colonne].median()

        print(f"Coefficient d'asymétrie de skewness pour {colonne}: {skewness} , Méthode de remplacement utilisée : {methode}")

        # Remplacer les valeurs manquantes
        dataframe.loc[:, colonne] = dataframe[colonne].fillna(valeur_remplacement)

    return dataframe

## Fonction pour remplacer les NA d'une colonne par regression linéaire
def remplacerNaParRegression(data, colonne_a_predire, colonnes_predictives):

    # Sélection des données d'entrainement et de test
    data_train = data[data[colonne_a_predire].notnull()]
    data_test = data[data[colonne_a_predire].isnull()]

    # Création et entrainement du modèle de régression linéaire
    model = LinearRegression()
    model.fit(data_train[colonnes_predictives], data_train[colonne_a_predire])

    # Prédiction des valeurs manquantes
    valeurs_prédites = model.predict(data_test[colonnes_predictives])

    # Remplacement des valeurs manquantes par celles prédites
    data.loc[data[colonne_a_predire].isnull(), colonne_a_predire] = valeurs_prédites

    return data
```

In [428…

```python
# Fonctions pour gerer les valeurs aberrantes

## Remplacer les valeurs maximales supérieures à 3 fois l'écart-type par la médiane
def remplacerValeursMaxParMediane(dataframe, colonnes):
    for colonne in colonnes:
        # Calculer la moyenne et l'écart-type de la colonne
        moyenne = dataframe[colonne].mean()
        ecart_type = dataframe[colonne].std()

        # Définir une valeur seuil (Les valeurs qui se trouvent à plus de trois écarts-types de la moyenne sont considérées comme des valeurs aberrantes)
        seuil = moyenne + 3 * ecart_type

        # Calculer la médiane de la colonne
        mediane = dataframe[colonne].median()

        # Remplacer les valeurs supérieures au seuil par la médiane
        dataframe.loc[dataframe[colonne] > seuil, colonne] = mediane

    return dataframe

## Fonction pour remplacer les valeurs trop petites par la médiane
def remplacerValeursMinParMediane(dataframe, colonnes):
    for colonne in colonnes:
        # Calculer la moyenne et l'écart-type de la colonne
        moyenne = dataframe[colonne].mean()
        ecart_type = dataframe[colonne].std()

        # Définir une valeur seuil (Les valeurs qui se trouvent à plus de trois écarts-types de la moyenne sont considérées comme des valeurs aberrantes)
        seuil = moyenne + 3 * ecart_type

        # Calculer la médiane de la colonne
        mediane = dataframe[colonne].median()

        # Remplacer les valeurs supérieures au seuil par la médiane
        dataframe.loc[dataframe[colonne] < seuil, colonne] = mediane
    return dataframe
```

In [429…

```python
# Fonctions pour visualiser les données

## Fonction pour visualiser les valeurs manquantes
def visuelDonneesNa(df):
    msno.bar(df,  color= 'dodgerblue')

## Fonction qui renvoie un histogramme et un boxplot pour chaque variable quantitative
def histoBoxPlotQuanti(data, colonnes):
    for col in colonnes:
        plt.figure(figsize=(10, 4))

        # Histogramme avec estimation de la densité de noyau (Kernel Density Estimate)
        plt.subplot(1, 2, 1)
        sns.histplot(data[col], kde=True)
        plt.title(f'Histogramme de {col}')
        plt.xlabel(col)
        plt.ylabel('Fréquence')

        # Boxplot
        plt.subplot(1, 2, 2)
        sns.boxplot(y=data[col])
        plt.title(f'Boxplot de {col}')
        plt.ylabel(col)

        plt.tight_layout()
        plt.show()
```

In [430…

```python
#Importation du fichier
fichier = importFichier("2016_Building_Energy_Benchmarking.csv")
fichier.head()
```

```
Chargement d'un fichier avec pour séparateur ','.
```

Out[430…

	OSEBuildingID	DataYear	BuildingType	PrimaryPropertyType	PropertyName	Address	City	State	ZipCode	TaxParcelIdentificationNumber	CouncilDistrictCode	Neighborhood	Latitude	Longitude	YearBuilt
0	1	2016	NonResidential	Hotel	Mayflower park hotel	405 Olive way	Seattle	WA	98101.0	0659000030	7	DOWNTOWN	47.61220	-122.33799	1927
1	2	2016	NonResidential	Hotel	Paramount Hotel	724 Pine street	Seattle	WA	98101.0	0659000220	7	DOWNTOWN	47.61317	-122.33393	1996
2	3	2016	NonResidential	Hotel	5673-The Westin Seattle	1900 5th Avenue	Seattle	WA	98101.0	0659000475	7	DOWNTOWN	47.61393	-122.33810	1969
3	5	2016	NonResidential	Hotel	HOTEL MAX	620 STEWART ST	Seattle	WA	98101.0	0659000640	7	DOWNTOWN	47.61412	-122.33664	1926
4	8	2016	NonResidential	Hotel	WARWICK SEATTLE HOTEL (ID8)	401 LENORA ST	Seattle	WA	98121.0	0659000970	7	DOWNTOWN	47.61375	-122.34047	1980

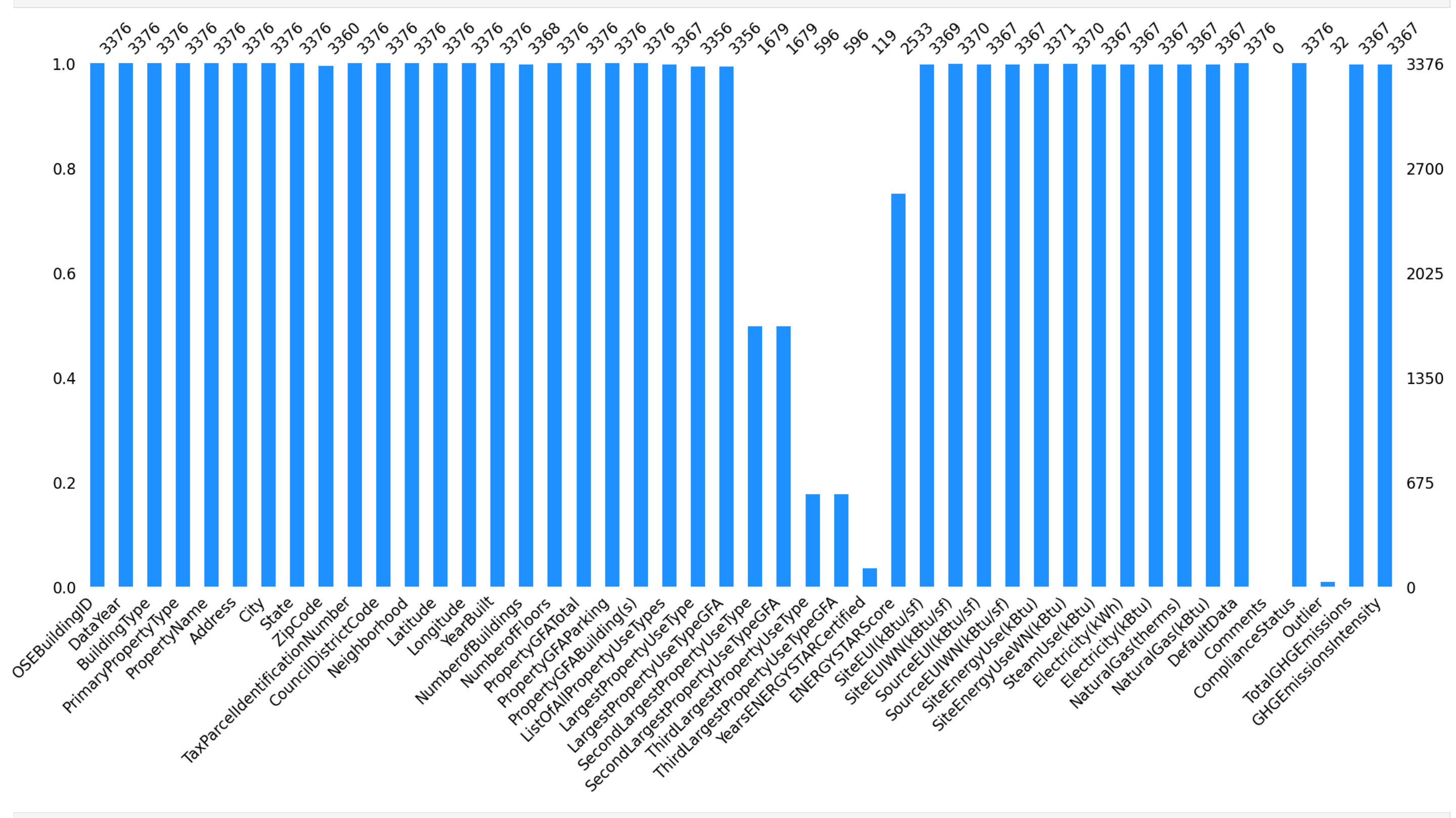
In [431…

```python
descriptionData(fichier)
```

```
La dataframe contient 3376 lignes et 46 colonnes.
```

In [432…

```python
#Visualisation des valeurs manquantes
visuelDonneesNa(fichier)
```



In [433…

```python
# Sélection des colonnes avec un taux de remplissage < 50%
col_na = colManquantes(fichier, 50)
col_na
```

```
nombre de colonnes avec un taux de remplissage inférieur à 50% : 7
```

Out[433…

```
['SecondLargestPropertyUseType',
 'SecondLargestPropertyUseTypeGFA',
 'ThirdLargestPropertyUseType',
 'ThirdLargestPropertyUseTypeGFA',
 'YearsENERGYSTARCertified',
 'Comments',
 'Outlier']
```

In [434…

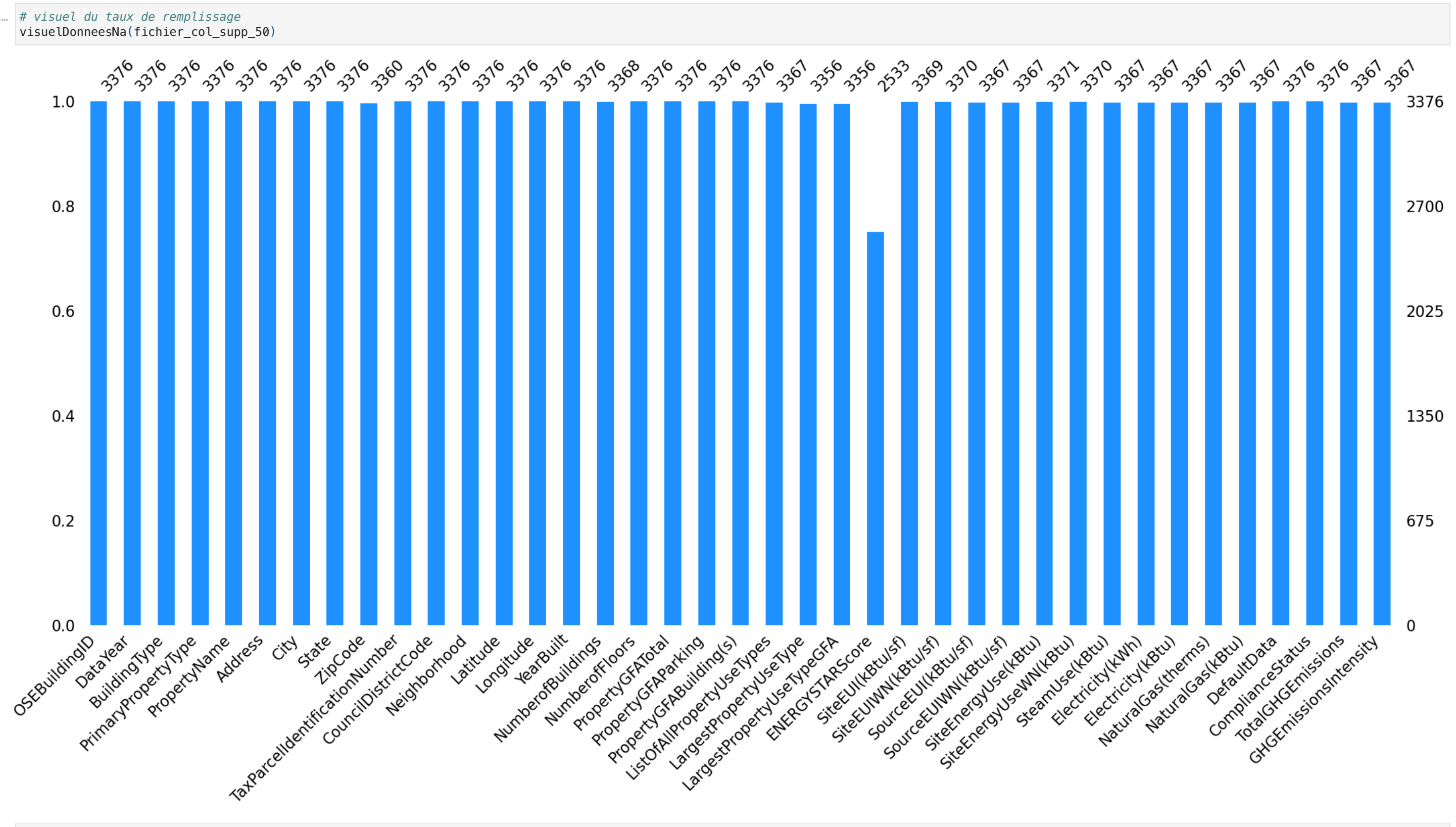
```python
# Suppression des colonnes avec un taux de remplissage < 50%
fichier_col_supp_50 = supprimeCol(fichier, col_na)
fichier_col_supp_50.head()
```

Out[434…

	OSEBuildingID	DataYear	BuildingType	PrimaryPropertyType	PropertyName	Address	City	State	ZipCode	TaxParcelIdentificationNumber	CouncilDistrictCode	Neighborhood	Latitude	Longitude	YearBuilt
0	1	2016	NonResidential	Hotel	Mayflower park hotel	405 Olive way	Seattle	WA	98101.0	0659000030	7	DOWNTOWN	47.61220	-122.33799	1927
1	2	2016	NonResidential	Hotel	Paramount Hotel	724 Pine street	Seattle	WA	98101.0	0659000220	7	DOWNTOWN	47.61317	-122.33393	1996
2	3	2016	NonResidential	Hotel	5673-The Westin Seattle	1900 5th Avenue	Seattle	WA	98101.0	0659000475	7	DOWNTOWN	47.61393	-122.33810	1969
3	5	2016	NonResidential	Hotel	HOTEL MAX	620 STEWART ST	Seattle	WA	98101.0	0659000640	7	DOWNTOWN	47.61412	-122.33664	1926
4	8	2016	NonResidential	Hotel	WARWICK SEATTLE HOTEL (ID8)	401 LENORA ST	Seattle	WA	98121.0	0659000970	7	DOWNTOWN	47.61375	-122.34047	1980

In [435…

```python
# visuel du taux de remplissage
visuelDonneesNa(fichier_col_supp_50)
```



In [436…

```python
# Premier choix et description des varialbes
# Liste des noms des colonnes sélectionnées
variables_list = ['OSEBuildingID', 'BuildingType', 'PrimaryPropertyType', 'YearBuilt', 'NumberofBuildings', 'NumberofFloors',
                  'PropertyGFATotal', 'PropertyGFAParking', 'PropertyGFABuilding(s)', 'SiteEUI(kBtu/sf)', 'SiteEUIWN(kBtu/sf)',
                  'SourceEUI(kBtu/sf)', 'SourceEUIWN(kBtu/sf)', 'SiteEnergyUse(kBtu)', 'SiteEnergyUseWN(kBtu)', 'SteamUse(kBtu)',
                  'Electricity(kWh)', 'Electricity(kBtu)', 'NaturalGas(therms)', 'NaturalGas(kBtu)', 'GHGEmissionsIntensity',
                  'ENERGYSTARScore']

# Liste des descriptions correspondantes
descriptions = ['ID du bâtiment OSE', 'Type de bâtiment (résidentiel, commercial, etc.)', 'Type de propriété principale', 'Année de construction', 'Nombre de bâtiments',
                'Nombre d\'étages', 'Surface totale du bâtiment et du parking', 'Surface de stationnement de la propriété', 'Surface du bâtiment de la propriété',
                'utilisation de l\'énergie du site d\'une propriété divisée par sa superficie brute au sol', 'EUI du site normalisé (énergétique du site que la propriété aurait consommée pendant les co
                'énergie de source d\'une propriété divisée par sa surface brute au sol', 'EUI de la source normalisé (kBtu/sf)',
                'Utilisation d\'énergie du site (kBtu)', 'Utilisation d\'énergie du site normalisé (kBtu)', 'La quantité annuelle de vapeur de district consommée par la propriété sur place, mesurée en
                'La quantité annuelle d\'électricité consommée par la propriété', 'La quantité annuelle d\'électricité consommée par la propriété', 'La quantité annuelle de gaz naturel consommée par la
                'Intensité des émissions de GES (kgCO2e/ft2)', 'Score ENERGY STAR']

# Création de la DataFrame
df_description = pd.DataFrame({
    'Nom de la colonne': variables_list,
    'Description': descriptions
})

# Ajuster les paramètres d'affichage de pandas pour afficher toutes les colonnes
pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', None)

# Affichage de la DataFrame
df_description
```

Out[436…

	Nom de la colonne	Description
0	OSEBuildingID	ID du bâtiment OSE
1	BuildingType	Type de bâtiment (résidentiel, commercial, etc.)
2	PrimaryPropertyType	Type de propriété principale
3	YearBuilt	Année de construction
4	NumberofBuildings	Nombre de bâtiments
5	NumberofFloors	Nombre d'étages
6	PropertyGFATotal	Surface totale du bâtiment et du parking
7	PropertyGFAParking	Surface de stationnement de la propriété
8	PropertyGFABuilding(s)	Surface du bâtiment de la propriété
9	SiteEUI(kBtu/sf)	utilisation de l'énergie du site d'une propriété divisée par sa superficie brute au sol
10	SiteEUIWN(kBtu/sf)	EUI du site normalisé (énergétique du site que la propriété aurait consommée pendant les conditions météorologiques moyennes de 30 ans)
11	SourceEUI(kBtu/sf)	énergie de source d'une propriété divisée par sa surface brute au sol
12	SourceEUIWN(kBtu/sf)	EUI de la source normalisé (kBtu/sf)
13	SiteEnergyUse(kBtu)	Utilisation d'énergie du site (kBtu)
14	SiteEnergyUseWN(kBtu)	Utilisation d'énergie du site normalisé (kBtu)
15	SteamUse(kBtu)	La quantité annuelle de vapeur de district consommée par la propriété sur place, mesurée en milliers d'unités thermiques britanniques (kBtu)
16	Electricity(kWh)	La quantité annuelle d'électricité consommée par la propriété
17	Electricity(kBtu)	La quantité annuelle d'électricité consommée par la propriété
18	NaturalGas(therms)	La quantité annuelle de gaz naturel consommée par la propriété (therms)
19	NaturalGas(kBtu)	La quantité annuelle de gaz naturel consommée par la propriété (kBtu)
20	GHGEmissionsIntensity	Intensité des émissions de GES (kgCO2e/ft2)
21	ENERGYSTARScore	Score ENERGY STAR

In [437…

```python
# Sélection des varialbes
data_select = colSelection(fichier_col_supp_50, variables_list)
data_select.head()
```

Out[437…

	OSEBuildingID	BuildingType	PrimaryPropertyType	YearBuilt	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)
0	1	NonResidential	Hotel	1927	1.0	12	88434	0	88434	81.699997	84.300003	182.500000
1	2	NonResidential	Hotel	1996	1.0	11	103566	15064	88502	94.800003	97.900002	176.100006
2	3	NonResidential	Hotel	1969	1.0	41	956110	196718	759392	96.000000	97.699997	241.899994
3	5	NonResidential	Hotel	1926	1.0	10	61320	0	61320	110.800003	113.300003	216.199997
4	8	NonResidential	Hotel	1980	1.0	18	175580	62000	113580	114.800003	118.699997	211.399994

In [438…

```python
# Nombre bâtiment par type
data_select["BuildingType"].value_counts()
```

Out[438…

```
BuildingType
NonResidential          1460
Multifamily LR (1-4)    1018
Multifamily MR (5-9)     580
Multifamily HR (10+)     110
SPS-District K-12         98
Nonresidential COS        85
Campus                    24
Nonresidential WA          1
Name: count, dtype: int64
```

In [439…

```python
# Selection des bâtiments contenant le type "NonResidential"
data_select_filtre = data_select[data_select["BuildingType"].str.contains("(?i)NonResidential", na=False)]
data_select_filtre.head()
```

Out[439…

	OSEBuildingID	BuildingType	PrimaryPropertyType	YearBuilt	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)
0	1	NonResidential	Hotel	1927	1.0	12	88434	0	88434	81.699997	84.300003	182.500000
1	2	NonResidential	Hotel	1996	1.0	11	103566	15064	88502	94.800003	97.900002	176.100006
2	3	NonResidential	Hotel	1969	1.0	41	956110	196718	759392	96.000000	97.699997	241.899994
3	5	NonResidential	Hotel	1926	1.0	10	61320	0	61320	110.800003	113.300003	216.199997
4	8	NonResidential	Hotel	1980	1.0	18	175580	62000	113580	114.800003	118.699997	211.399994

In [440…

```python
# Nombre bâtiment par type (vérification)
data_select_filtre["BuildingType"].value_counts()
```

Out[440…

```
BuildingType
NonResidential        1460
Nonresidential COS      85
Nonresidential WA        1
Name: count, dtype: int64
```

In [441…

```python
# Conversion de l'id des batiments et de l'année de conctruction en facteur
data_select_filtre[["OSEBuildingID", "YearBuilt"]] = data_select_filtre[["OSEBuildingID", "YearBuilt"]].astype('category')
```

```
/var/folders/h0/8500ftrn70xcn1_yvyhmwns00000gn/T/ipykernel_12417/3836959709.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data_select_filtre[["OSEBuildingID", "YearBuilt"]] = data_select_filtre[["OSEBuildingID", "YearBuilt"]].astype('category')
```

In [442…

```python
data_select_filtre.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1546 entries, 0 to 3375
Data columns (total 22 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   OSEBuildingID           1546 non-null   category
 1   BuildingType            1546 non-null   object
 2   PrimaryPropertyType     1546 non-null   object
 3   YearBuilt               1546 non-null   category
 4   NumberofBuildings       1544 non-null   float64
 5   NumberofFloors          1546 non-null   int64
 6   PropertyGFATotal        1546 non-null   int64
 7   PropertyGFAParking      1546 non-null   int64
 8   PropertyGFABuilding(s)  1546 non-null   int64
 9   SiteEUI(kBtu/sf)        1543 non-null   float64
 10  SiteEUIWN(kBtu/sf)      1543 non-null   float64
 11  SourceEUI(kBtu/sf)      1544 non-null   float64
 12  SourceEUIWN(kBtu/sf)    1544 non-null   float64
 13  SiteEnergyUse(kBtu)     1544 non-null   float64
 14  SiteEnergyUseWN(kBtu)   1543 non-null   float64
 15  SteamUse(kBtu)          1544 non-null   float64
 16  Electricity(kWh)        1544 non-null   float64
 17  Electricity(kBtu)       1544 non-null   float64
 18  NaturalGas(therms)      1544 non-null   float64
 19  NaturalGas(kBtu)        1544 non-null   float64
 20  GHGEmissionsIntensity   1544 non-null   float64
 21  ENERGYSTARScore         1006 non-null   float64
dtypes: category(2), float64(14), int64(4), object(2)
memory usage: 307.5+ KB
```

In [443…

```python
# Catégorisation des variables
quali,quanti = categoriseVariables(data_select_filtre)
```

In [444…

```python
# variables qualitatives
quali
```

Out[444…

```
['OSEBuildingID', 'BuildingType', 'PrimaryPropertyType', 'YearBuilt']
```

In [445…

```python
# variables quantitatives
quanti
```

Out[445…

```
['NumberofBuildings',
 'NumberofFloors',
 'PropertyGFATotal',
 'PropertyGFAParking',
 'PropertyGFABuilding(s)',
 'SiteEUI(kBtu/sf)',
 'SiteEUIWN(kBtu/sf)',
 'SourceEUI(kBtu/sf)',
 'SourceEUIWN(kBtu/sf)',
 'SiteEnergyUse(kBtu)',
 'SiteEnergyUseWN(kBtu)',
 'SteamUse(kBtu)',
 'Electricity(kWh)',
 'Electricity(kBtu)',
 'NaturalGas(therms)',
 'NaturalGas(kBtu)',
 'GHGEmissionsIntensity',
 'ENERGYSTARScore']
```

In [446…

```python
#histoBoxPlotQuanti(fichier_col_supp_50, quanti)
```

In [447…

```python
# verification des doublons
data_select_filtre["OSEBuildingID"].duplicated().sum()
```

Out[447…

```
0
```

In [448…

```python
data_select_filtre.describe()
```

Out[448…

	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)	SiteEnergyUse(kBtu)
count	1544.000000	1546.000000	1.546000e+03	1546.000000	1.546000e+03	1543.000000	1543.000000	1544.000000	1544.000000	1.544000e+03
mean	1.036269	4.283959	1.118044e+05	13822.981889	9.798146e+04	74.203953	76.585677	181.615026	183.843588	7.642371e+06
std	0.569231	6.783014	1.915023e+05	43679.918511	1.682204e+05	75.588468	76.576520	190.144184	190.131202	1.865844e+07
min	0.000000	0.000000	1.128500e+04	0.000000	3.636000e+03	0.000000	0.000000	0.000000	-2.100000	0.000000e+00
25%	1.000000	1.000000	2.877000e+04	0.000000	2.770000e+04	34.250000	36.099998	78.699997	81.000000	1.204670e+06
50%	1.000000	2.000000	4.730250e+04	0.000000	4.511350e+04	53.200001	55.799999	137.750000	140.099998	2.609590e+06
75%	1.000000	4.000000	1.029090e+05	0.000000	9.261400e+04	83.750000	87.400002	210.375004	210.624996	6.968646e+06
max	9.000000	99.000000	1.952220e+06	512608.000000	1.765970e+06	834.400024	834.400024	2620.000000	2620.000000	2.930908e+08

In [449…

```python
# Filtre des valeurs inférieures à 0
## selectionner les colonnes quantitatives
data_quanti = data_select_filtre[quanti]
## Filtrer les lignes où les valeurs des colonnes quantitatives sont inférieures à 0
negative_quanti_val = data_quanti[(data_quanti < 0).any(axis = 1)]
negative_quanti_val
```

Out[449…

	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)	SiteEnergyUse(kBtu)
3206	1.0	6	52000	0	52000	7.1	5.0	2.0	-2.1	342726.0938

In [450…

```python
# Suppression des valeurs inférieures à 0
data_select_filtre = data_select_filtre.drop(negative_quanti_val.index).reset_index(drop=True)
data_select_filtre.head()
```

Out[450…

	OSEBuildingID	BuildingType	PrimaryPropertyType	YearBuilt	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)
0	1	NonResidential	Hotel	1927	1.0	12	88434	0	88434	81.699997	84.300003	182.500000
1	2	NonResidential	Hotel	1996	1.0	11	103566	15064	88502	94.800003	97.900002	176.100006
2	3	NonResidential	Hotel	1969	1.0	41	956110	196718	759392	96.000000	97.699997	241.899994
3	5	NonResidential	Hotel	1926	1.0	10	61320	0	61320	110.800003	113.300003	216.199997
4	8	NonResidential	Hotel	1980	1.0	18	175580	62000	113580	114.800003	118.699997	211.399994

In [451…

```python
# Choix des variables finales
quali = ['OSEBuildingID', 'YearBuilt']
quanti = ['NumberofBuildings', 'NumberofFloors', 'PropertyGFATotal', 'PropertyGFAParking', 'PropertyGFABuilding(s)', 'SiteEUI(kBtu/sf)', 'SiteEUIWN(kBtu/sf)',
          'SourceEUI(kBtu/sf)', 'SourceEUIWN(kBtu/sf)', 'SiteEnergyUse(kBtu)', 'SiteEnergyUseWN(kBtu)', 'SteamUse(kBtu)', 'Electricity(kBtu)', 'NaturalGas(kBtu)', 'GHGEmissionsIntensity', 'ENERGYSTARSc

# Dataframe finale
data_final = data_select_filtre[quali + quanti]
data_final.head()
```

Out[451…

	OSEBuildingID	YearBuilt	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)
0	1	1927	1.0	12	88434	0	88434	81.699997	84.300003	182.500000	189.000000
1	2	1996	1.0	11	103566	15064	88502	94.800003	97.900002	176.100006	179.399994
2	3	1969	1.0	41	956110	196718	759392	96.000000	97.699997	241.899994	244.100006
3	5	1926	1.0	10	61320	0	61320	110.800003	113.300003	216.199997	224.000000
4	8	1980	1.0	18	175580	62000	113580	114.800003	118.699997	211.399994	215.600006

In [452…

```python
# Description de la dataframe finale
descriptionData(data_final)
```

```
La dataframe contient 1545 lignes et 18 colonnes.
```

In [453…

```python
# Description des variables finales
data_final_describe = df_description[df_description['Nom de la colonne'].isin(data_final.columns)].reset_index(drop=True)
data_final_describe
```

Out[453…

	Nom de la colonne	Description
0	OSEBuildingID	ID du bâtiment OSE
1	YearBuilt	Année de construction
2	NumberofBuildings	Nombre de bâtiments
3	NumberofFloors	Nombre d'étages
4	PropertyGFATotal	Surface totale du bâtiment et du parking
5	PropertyGFAParking	Surface de stationnement de la propriété
6	PropertyGFABuilding(s)	Surface du bâtiment de la propriété
7	SiteEUI(kBtu/sf)	utilisation de l'énergie du site d'une propriété divisée par sa superficie brute au sol
8	SiteEUIWN(kBtu/sf)	EUI du site normalisé (énergétique du site que la propriété aurait consommée pendant les conditions météorologiques moyennes de 30 ans)
9	SourceEUI(kBtu/sf)	énergie de source d'une propriété divisée par sa surface brute au sol
10	SourceEUIWN(kBtu/sf)	EUI de la source normalisé (kBtu/sf)
11	SiteEnergyUse(kBtu)	Utilisation d'énergie du site (kBtu)
12	SiteEnergyUseWN(kBtu)	Utilisation d'énergie du site normalisé (kBtu)
13	SteamUse(kBtu)	La quantité annuelle de vapeur de district consommée par la propriété sur place, mesurée en milliers d'unités thermiques britanniques (kBtu)
14	Electricity(kBtu)	La quantité annuelle d'électricité consommée par la propriété
15	NaturalGas(kBtu)	La quantité annuelle de gaz naturel consommée par la propriété (kBtu)
16	GHGEmissionsIntensity	Intensité des émissions de GES (kgCO2e/ft2)
17	ENERGYSTARScore	Score ENERGY STAR

In [454…

```python
# Vérification des valeurs manquantes
data_final.isnull().sum()
```

Out[454…

```
OSEBuildingID              0
YearBuilt                  0
NumberofBuildings          2
NumberofFloors             0
PropertyGFATotal           0
PropertyGFAParking         0
PropertyGFABuilding(s)     0
SiteEUI(kBtu/sf)           3
SiteEUIWN(kBtu/sf)         3
SourceEUI(kBtu/sf)         2
SourceEUIWN(kBtu/sf)       2
SiteEnergyUse(kBtu)        2
SiteEnergyUseWN(kBtu)      3
SteamUse(kBtu)             2
Electricity(kBtu)          2
NaturalGas(kBtu)           2
GHGEmissionsIntensity      2
ENERGYSTARScore          540
dtype: int64
```

In [455…

```python
# Remplacement des valeurs manquantes par la methode statistique appropriée
## selection des colonnes quantitatives avec des valeurs manquantes a remplacer
quanti_select = ['NumberofBuildings', 'NumberofFloors', 'PropertyGFATotal', 'PropertyGFAParking', 'PropertyGFABuilding(s)', 'SiteEUI(kBtu/sf)', 'SiteEUIWN(kBtu/sf)',
         'SourceEUI(kBtu/sf)', 'SourceEUIWN(kBtu/sf)', 'SiteEnergyUse(kBtu)', 'SiteEnergyUseWN(kBtu)', 'SteamUse(kBtu)', 'Electricity(kBtu)', 'NaturalGas(kBtu)', 'GHGEmissionsIntensity']
data_final = remplacerValParStats(data_final, quanti_select)
```

```
Coefficient d'asymétrie de skewness pour NumberofBuildings: 7.99549171350431 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour NumberofFloors: 5.833270323672367 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour PropertyGFATotal: 4.729565425456481 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour PropertyGFAParking: 5.033754800476555 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour PropertyGFABuilding(s): 5.003158643035811 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour SiteEUI(kBtu/sf): 3.6828967727814805 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour SiteEUIWN(kBtu/sf): 3.587332760117949 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour SourceEUI(kBtu/sf): 4.8827159244462885 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour SourceEUIWN(kBtu/sf): 4.860137266291067 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour SiteEnergyUse(kBtu): 9.415287792099788 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour SiteEnergyUseWN(kBtu): 9.480567739199575 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour SteamUse(kBtu): 20.54358651123535 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour Electricity(kBtu): 9.507962370510231 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour NaturalGas(kBtu): 14.257942197856007 , Méthode de remplacement utilisée : médiane
Coefficient d'asymétrie de skewness pour GHGEmissionsIntensity: 3.693560344204123 , Méthode de remplacement utilisée : médiane
```

In [456…

```python
# Revérification des valeurs manquantes
data_final.isnull().sum()
```

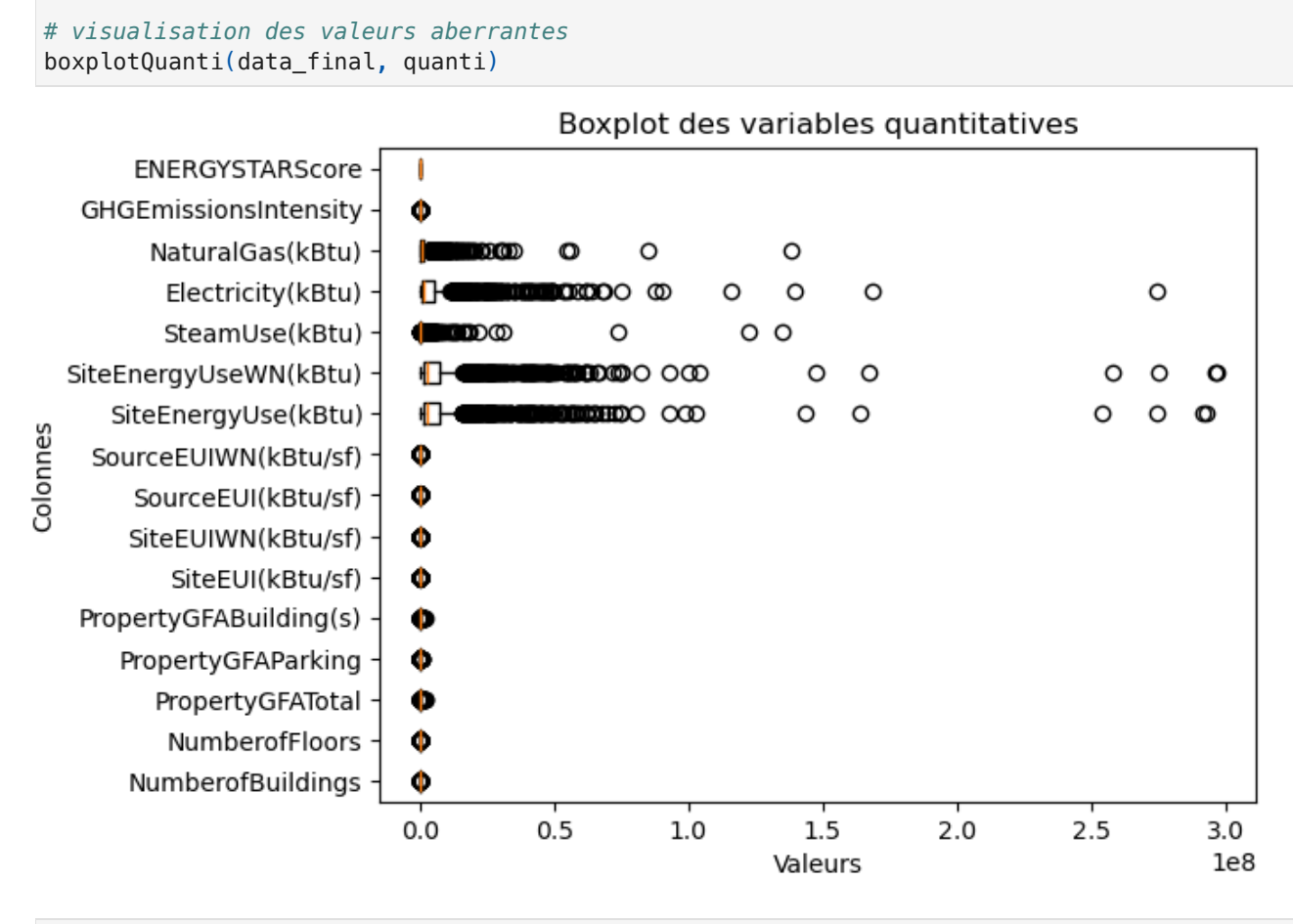
Out[456…

```
OSEBuildingID              0
YearBuilt                  0
NumberofBuildings          0
NumberofFloors             0
PropertyGFATotal           0
PropertyGFAParking         0
PropertyGFABuilding(s)     0
SiteEUI(kBtu/sf)           0
SiteEUIWN(kBtu/sf)         0
SourceEUI(kBtu/sf)         0
SourceEUIWN(kBtu/sf)       0
SiteEnergyUse(kBtu)        0
SiteEnergyUseWN(kBtu)      0
SteamUse(kBtu)             0
Electricity(kBtu)          0
NaturalGas(kBtu)           0
GHGEmissionsIntensity      0
ENERGYSTARScore          540
dtype: int64
```

In [457…

```python
# fonction qui retourne un boxplot des valeurs aberrantes des variables quantitatives
def boxplotQuanti(data, colonnes):
    plt.boxplot([data[col].dropna() for col in colonnes ], labels= colonnes, vert= False)
    plt.title("Boxplot des variables quantitatives")
    plt.xlabel("Valeurs")
    plt.ylabel("Colonnes")
    plt.show()

# visualisation des valeurs aberrantes
boxplotQuanti(data_final, quanti)
```



In [458…

```python
# Description des variables quantitatives
data_final.describe()
```

Out[458…

	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)	SiteEnergyUse(kBtu)
count	1545.000000	1545.000000	1.545000e+03	1545.000000	1.545000e+03	1545.000000	1545.000000	1545.000000	1545.000000	1.545000e+03
mean	1.036246	4.282848	1.118432e+05	13831.928803	9.801122e+04	74.206602	76.591651	181.674693	183.907443	7.640588e+06
std	0.569047	6.785070	1.915582e+05	43692.644201	1.682708e+05	75.525864	76.510706	190.034133	190.017186	1.865235e+07
min	0.000000	0.000000	1.128500e+04	0.000000	3.636000e+03	0.000000	0.000000	0.000000	0.000000	0.000000e+00
25%	1.000000	1.000000	2.876000e+04	0.000000	2.769000e+04	34.299999	36.200001	79.300003	81.300003	1.207504e+06
50%	1.000000	2.000000	4.723700e+04	0.000000	4.508200e+04	53.200001	55.799999	137.899994	140.199997	2.615248e+06
75%	1.000000	4.000000	1.029300e+05	0.000000	9.279200e+04	83.699997	87.400002	210.300003	210.399994	6.963616e+06
max	9.000000	99.000000	1.952220e+06	512608.000000	1.765970e+06	834.400024	834.400024	2620.000000	2620.000000	2.930908e+08

In [459…

```python
# Remplacement des valeurs trop grandes (supérieures à 3 fois l'écart-type) par la médiane
data_final = remplacerValeursMaxParMediane(data_final, quanti_select)
```

In [460…

```python
data_final.describe()
```

Out[460…

	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)	SiteEnergyUse(kBtu)
count	1545.000000	1545.000000	1545.000000	1545.000000	1545.000000	1545.000000	1545.000000	1545.000000	1545.000000	1.545000e+03
mean	0.975405	3.455016	86821.899676	7502.137864	76570.703560	67.128997	69.281683	166.996505	168.845243	6.150506e+06
std	0.205282	3.472983	101970.518499	21750.567712	87768.246638	53.705992	54.390423	133.127162	132.488583	9.415650e+06
min	0.000000	0.000000	11285.000000	0.000000	3636.000000	0.000000	0.000000	0.000000	0.000000	0.000000e+00
25%	1.000000	1.000000	28760.000000	0.000000	27690.000000	34.299999	36.200001	79.300003	81.300003	1.207504e+06
50%	1.000000	2.000000	47237.000000	0.000000	45082.000000	53.200001	55.799999	137.899994	140.199997	2.615248e+06
75%	1.000000	4.000000	95117.000000	0.000000	86204.000000	80.300003	83.500000	202.899994	204.699997	6.674180e+06
max	2.000000	24.000000	635824.000000	141600.000000	597519.000000	298.299988	303.799988	747.700012	745.400024	6.219718e+07

In [461…

```python
# Nombre de valeurs À 0 présentes par colonne
data_final[data_final == 0].count()
```

Out[461…

```
OSEBuildingID                0
YearBuilt                    0
NumberofBuildings           52
NumberofFloors              14
PropertyGFATotal             0
PropertyGFAParking        1259
PropertyGFABuilding(s)       0
SiteEUI(kBtu/sf)             1
SiteEUIWN(kBtu/sf)          10
SourceEUI(kBtu/sf)           4
SourceEUIWN(kBtu/sf)        13
SiteEnergyUse(kBtu)          2
SiteEnergyUseWN(kBtu)       10
SteamUse(kBtu)            1442
Electricity(kBtu)            4
NaturalGas(kBtu)           459
GHGEmissionsIntensity        5
ENERGYSTARScore              0
dtype: int64
```

In [462…

```python
# Remplacement des valeurs trop petites (inférieur à 3 fois l'écart-type) par la médiane
data_final = remplacerValeursMinParMediane(data_final, quanti_select)
```

In [463…

```python
# Revérification du nombre de valeurs À 0 présentes par colonne
data_final[data_final == 0].count()
```

Out[463…

```
OSEBuildingID                0
YearBuilt                    0
NumberofBuildings            0
NumberofFloors               0
PropertyGFATotal             0
PropertyGFAParking        1489
PropertyGFABuilding(s)       0
SiteEUI(kBtu/sf)             0
SiteEUIWN(kBtu/sf)           0
SourceEUI(kBtu/sf)           0
SourceEUIWN(kBtu/sf)         0
SiteEnergyUse(kBtu)          0
SiteEnergyUseWN(kBtu)        0
SteamUse(kBtu)            1517
Electricity(kBtu)            0
NaturalGas(kBtu)             0
GHGEmissionsIntensity        0
ENERGYSTARScore              0
dtype: int64
```

In [464…

```python
descriptionData(data_final)
```

```
La dataframe contient 1545 lignes et 18 colonnes.
```

In [465…

```python
data_final["PropertyGFAParking"].value_counts()
```

Out[465…

```
PropertyGFAParking
0         1489
100176       2
85000        1
113318       1
111774       1
101342       1
94892        1
78848        1
80283        1
117867       1
88855        1
103650       1
84689        1
116280       1
89778        1
87160        1
74481        1
99314        1
124788       1
138098       1
112980       1
79284        1
100800       1
84710        1
112124       1
83064        1
114010       1
91935        1
124800       1
98556        1
97400        1
141600       1
84600        1
129432       1
82720        1
118744       1
116721       1
124216       1
83591        1
134662       1
86429        1
93204        1
104000       1
121375       1
104922       1
115228       1
111793       1
98700        1
95000        1
73673        1
109905       1
88405        1
95281        1
114829       1
103032       1
85028        1
Name: count, dtype: int64
```

In [466…

```python
data_final["SteamUse(kBtu)"].value_counts()
```

Out[466…

```
SteamUse(kBtu)
0.0           1517
10921066.0       1
13296249.0       1
8630381.0        1
5473370.5        1
4047481.0        1
7413886.0        1
6090187.5        1
7815663.0        1
5650343.5        1
15586014.0       1
16284570.0       1
12760971.0       1
5370612.0        1
6568832.0        1
5237165.5        1
4336524.5        1
4311136.0        1
5137220.0        1
9763690.0        1
4120130.0        1
6093150.0        1
5327802.5        1
11598686.0       1
4592347.5        1
4870847.5        1
4337738.5        1
5999360.5        1
13556219.0       1
Name: count, dtype: int64
```

In [467…

```python
#Fonction pour remplacer les 0 par la médiane
def remplacerZeroParMediane(dataframe, colonnes):
    for colonne in colonnes:
        mediane = dataframe[colonne].median()
        dataframe.loc[dataframe[colonne] == 0, colonne] = mediane
    return dataframe

# Remplacement des valeurs à 0 par la médiane
data_final = remplacerZeroParMediane(data_final, quanti_select)
```

In [468…

```python
# dernière vérification Nombre de valeurs À 0 présentes par colonne
data_final[data_final == 0].count()
```

Out[468…

```
OSEBuildingID                0
YearBuilt                    0
NumberofBuildings            0
NumberofFloors               0
PropertyGFATotal             0
PropertyGFAParking        1489
PropertyGFABuilding(s)       0
SiteEUI(kBtu/sf)             0
SiteEUIWN(kBtu/sf)           0
SourceEUI(kBtu/sf)           0
SourceEUIWN(kBtu/sf)         0
SiteEnergyUse(kBtu)          0
SiteEnergyUseWN(kBtu)        0
SteamUse(kBtu)            1517
Electricity(kBtu)            0
NaturalGas(kBtu)             0
GHGEmissionsIntensity        0
ENERGYSTARScore              0
dtype: int64
```

Les valeurs à 0 des colonnes PropertyGFAParking (Surface de stationnement de la propriété) et SteamUse (La quantité annuelle de vapeur de district consommée par la propriété) n'ont pas changés, celà semble bizare mais pas impossible donc on va les laisser telquel .

In [469…

```python
# Détection des valeurs abérrantes
## Methode de l'intervalle interquartile des variables quantitatives
def detectOutlierIqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    born_inf = Q1 - 1.5 * IQR
    born_sup = Q3 + 1.5 * IQR
    outliers = data[(data[column] < born_inf) | (data[column] > born_sup)]
    return outliers

outliers = detectOutlierIqr(data_final, "SiteEnergyUseWN(kBtu)")
outliers
```

Out[469…

	OSEBuildingID	YearBuilt	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)	SiteEnergyUs
19	26	1916	1.0	2	540360	0	540360	53.200001	55.799999	137.899994	140.199997	4498
20	27	1985	1.0	19	47237	0	385274	53.200001	55.799999	137.899994	140.199997	4785
30	46	1961	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4473
51	84	1922	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4279
63	107	2010	1.0	2	571329	0	571329	53.200001	55.799999	137.899994	140.199997	3960
68	114	2010	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	3666
81	147	1954	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261
114	245	1999	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	5975
117	249	1999	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	5876
134	295	1955	1.0	15	47237	0	45082	53.200001	303.799988	137.899994	140.199997	6219
147	327	1925	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4006
148	328	1976	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4084
156	336	2009	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	3952
163	345	1983	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4468
168	350	1958	1.0	2	47237	0	45082	297.399994	299.200012	137.899994	140.199997	4139
171	353	1969	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4516
172	354	2002	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4121
175	357	1990	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	6157
203	388	1976	1.0	2	635824	0	45082	53.200001	55.799999	137.899994	140.199997	4847
204	389	1923	1.0	2	542305	0	542305	53.200001	55.799999	137.899994	140.199997	5648
213	399	1988	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	5307
215	402	1989	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	5660
222	410	1973	1.0	19	47237	0	45082	53.200001	55.799999	137.899994	140.199997	3778
227	415	1985	1.0	2	47237	134662	45082	53.200001	55.799999	137.899994	140.199997	5222
233	422	1989	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4951
248	439	2001	1.0	2	47237	0	45082	283.700012	283.700012	137.899994	140.199997	5507
268	465	1929	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4840
269	466	1975	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	6176
285	488	2008	1.0	2	436497	0	45082	53.200001	55.799999	137.899994	140.199997	4525
294	498	2000	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4270
312	525	1991	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	5296
368	588	2002	1.0	2	47237	113318	45082	280.399994	284.700012	657.099976	661.700012	4710
413	637	2009	1.0	2	47237	111774	45082	53.200001	55.799999	137.899994	140.199997	261
426	657	2006	1.0	2	516407	0	516407	53.200001	55.799999	137.899994	140.199997	3940
428	659	2005	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	5649
485	743	1999	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	739.900024	3643
499	758	1999	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4651
505	764	1970	1.0	2	489182	0	489182	53.200001	55.799999	137.899994	140.199997	3637
512	772	1981	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	3798
536	803	1929	1.0	2	47237	0	45082	53.200001	55.799999	725.599976	727.200012	3795
868	21741	2006	1.0	14	47237	0	45082	53.200001	55.799999	137.899994	140.199997	3779
1464	45927	2000	1.0	2	47237	0	45082	292.200012	302.700012	587.200012	594.400024	5316
1465	48287	2005	1.0	2	47237	0	45082	251.600006	255.399994	718.799988	722.700012	4061
1476	49725	2012	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4258
1477	49732	2013	1.0	2	47237	0	45082	239.199997	239.199997	137.899994	140.199997	5776
1483	49780	1975	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	4168
1484	49783	2013	1.0	2	47237	114010	45082	240.899994	248.199997	137.899994	140.199997	4169
1499	49926	1925	1.0	2	428347	0	428347	53.200001	55.799999	137.899994	140.199997	3702
1527	50094	2015	1.0	2	47237	85028	45082	53.200001	55.799999	137.899994	140.199997	3936

In [470…

```python
# Description des valeurs abérrantes
descriptionData(outliers)
```

```
La dataframe contient 49 lignes et 18 colonnes.
```

In [471…

```python
# Suppression des valeurs aberrantes avec IQR
def remove_outliers(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return data[(data[column] >= lower_bound) & (data[column] <= upper_bound)]

# Exemple d'application
data_clean = remove_outliers(data_final, 'SiteEnergyUseWN(kBtu)')
```

In [472…

```python
data_clean["ENERGYSTARScore"].unique()
```

Out[472…

```
array([ 60.,  61.,  43.,  56.,  75.,  nan,  27.,  30.,  36.,  48.,  58.,
        35.,  31.,  39.,  84.,  57.,  29.,  83.,  95.,  67.,  78.,  14.,
        71.,  63.,  98.,  99.,  86.,  80.,  81.,  76.,  94.,   1.,  74.,
        96.,  37.,  68.,  53.,  34., 100.,  89.,  21.,  10.,  72.,  87.,
        79.,  70.,  64.,  91.,  49.,  62.,  46.,  42.,   8.,  20.,  90.,
        85.,  66.,  69.,  28.,  77.,  88.,  93.,  65.,   3.,  54.,  97.,
         2.,  55.,  25.,  45.,  59.,  19.,   7.,   5.,  38.,  73.,  33.,
        26.,  41.,  32.,  92.,  82.,  15.,  16.,   6.,  50.,  12.,  51.,
        13.,  11.,  47.,  18.,  24.,  17.,  52.,   4.,  23.,  44.,  40.,
        22.,   9.])
```

In [473…

```python
data_clean.head()
```

Out[473…

	OSEBuildingID	YearBuilt	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)	SiteEnergyUse(k
0	1	1927	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
1	2	1996	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
2	3	1969	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
3	5	1926	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
4	8	1980	1.0	18	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524

In [474…

```python
# Transformation des colonnes en proportions par rapport à la consommation totale d'énergie
def transformer_en_proportions(data):
    # Calcul de la consommation totale d'énergie
    data['TotalEnergy(kBtu)'] = data['Electricity(kBtu)'] + data['SteamUse(kBtu)'] + data['NaturalGas(kBtu)']

    # Transformation des colonnes en proportions
    data['Electricity_Proportion'] = data['Electricity(kBtu)'] / data['TotalEnergy(kBtu)']
    data['SteamUse_Proportion'] = data['SteamUse(kBtu)'] / data['TotalEnergy(kBtu)']
    data['NaturalGas_Proportion'] = data['NaturalGas(kBtu)'] / data['TotalEnergy(kBtu)']

    return data

# Application de la transformation
data_clean = transformer_en_proportions(data_clean)

# Affichage des premières lignes pour vérifier
data_clean[['Electricity_Proportion', 'SteamUse_Proportion', 'NaturalGas_Proportion']].head()
```

```
/var/folders/h0/8500ftrn70xcn1_yvyhmwns00000gn/T/ipykernel_12417/795362544.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['TotalEnergy(kBtu)'] = data['Electricity(kBtu)'] + data['SteamUse(kBtu)'] + data['NaturalGas(kBtu)']
/var/folders/h0/8500ftrn70xcn1_yvyhmwns00000gn/T/ipykernel_12417/795362544.py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['Electricity_Proportion'] = data['Electricity(kBtu)'] / data['TotalEnergy(kBtu)']
/var/folders/h0/8500ftrn70xcn1_yvyhmwns00000gn/T/ipykernel_12417/795362544.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['SteamUse_Proportion'] = data['SteamUse(kBtu)'] / data['TotalEnergy(kBtu)']
/var/folders/h0/8500ftrn70xcn1_yvyhmwns00000gn/T/ipykernel_12417/795362544.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['NaturalGas_Proportion'] = data['NaturalGas(kBtu)'] / data['TotalEnergy(kBtu)']
```

Out[474…

	Electricity_Proportion	SteamUse_Proportion	NaturalGas_Proportion
0	0.790551	0.0	0.209449
1	0.790551	0.0	0.209449
2	0.790551	0.0	0.209449
3	0.790551	0.0	0.209449
4	0.156597	0.0	0.843403

In [475…

```python
# Suppression des colonnes de consommation d'énergie brutes
data_clean = data_clean.drop(columns=['Electricity(kBtu)', 'SteamUse(kBtu)', 'NaturalGas(kBtu)', 'TotalEnergy(kBtu)'])
```

In [485…

```python
# dataframe finale
data_clean.head()
```

Out[485…

	OSEBuildingID	YearBuilt	NumberofBuildings	NumberofFloors	PropertyGFATotal	PropertyGFAParking	PropertyGFABuilding(s)	SiteEUI(kBtu/sf)	SiteEUIWN(kBtu/sf)	SourceEUI(kBtu/sf)	SourceEUIWN(kBtu/sf)	SiteEnergyUse(k
0	1	1927	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
1	2	1996	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
2	3	1969	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
3	5	1926	1.0	2	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524
4	8	1980	1.0	18	47237	0	45082	53.200001	55.799999	137.899994	140.199997	261524

Analyses

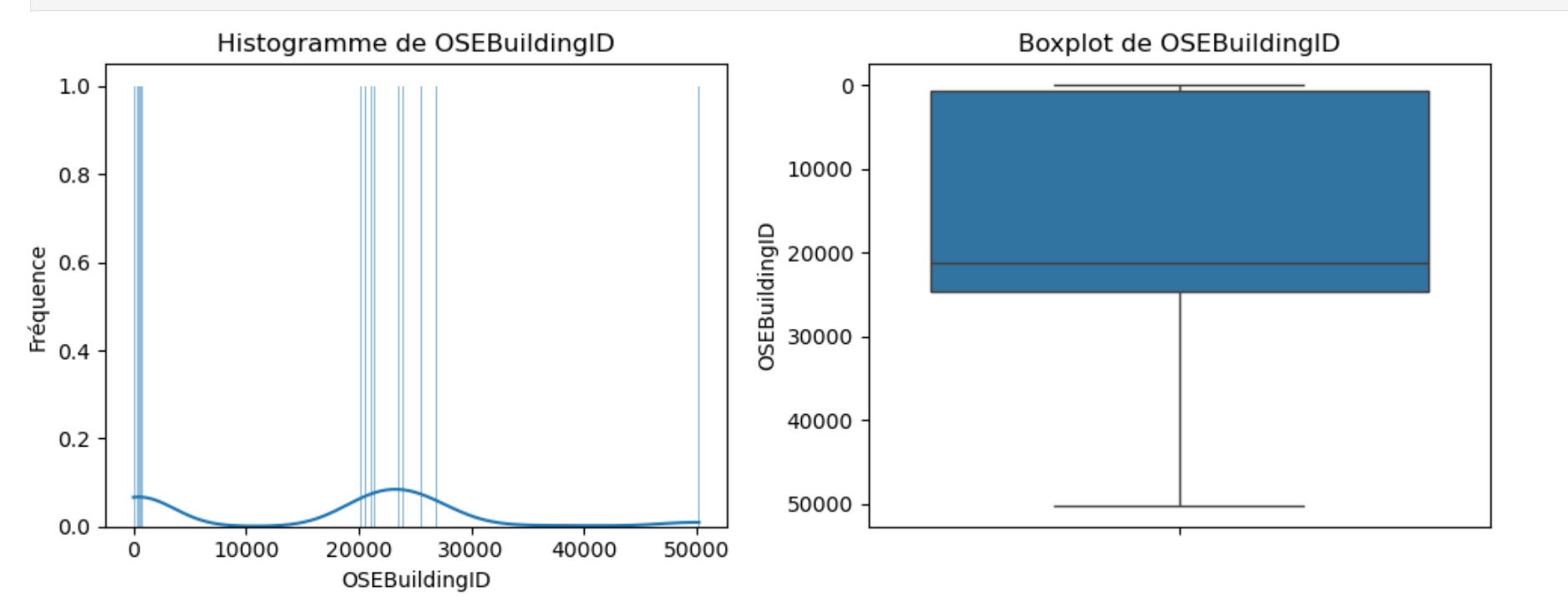
UNIVARIEE

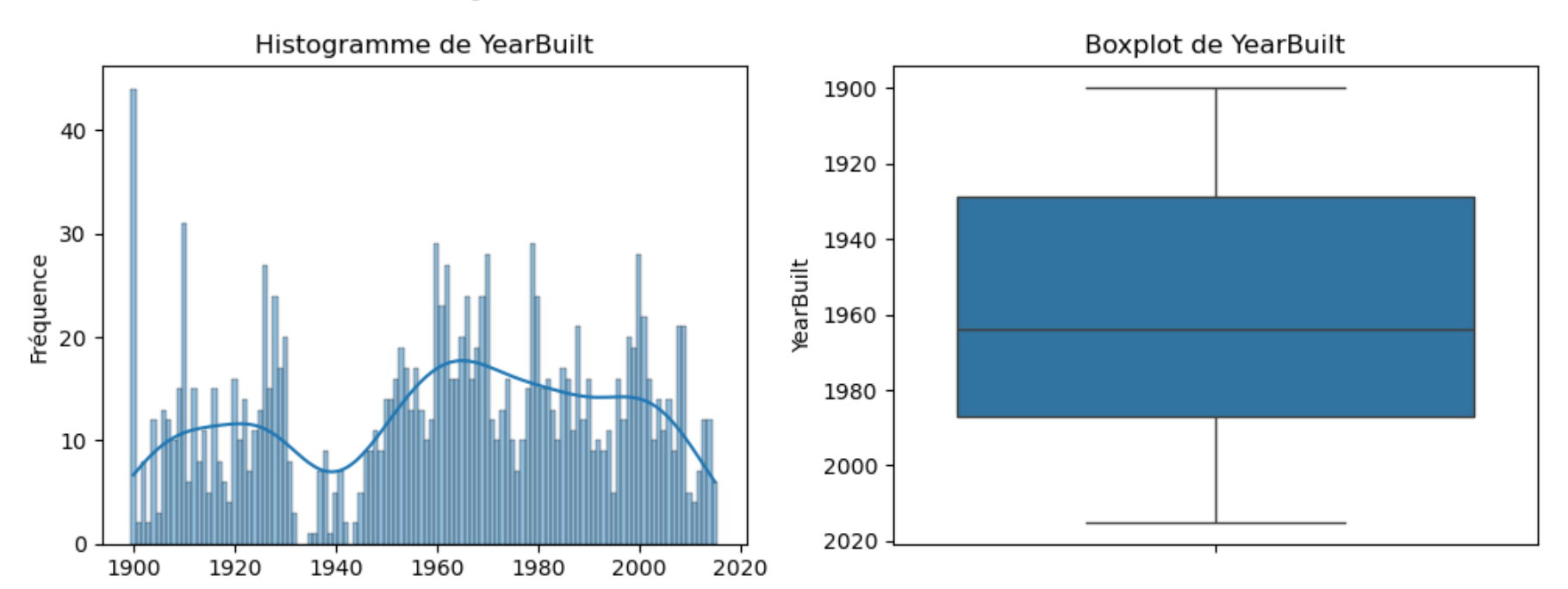
In [478…

```python
# Nouvelle atégorisation des variables
quali,quanti = categoriseVariables(data_clean)
```

In [479…

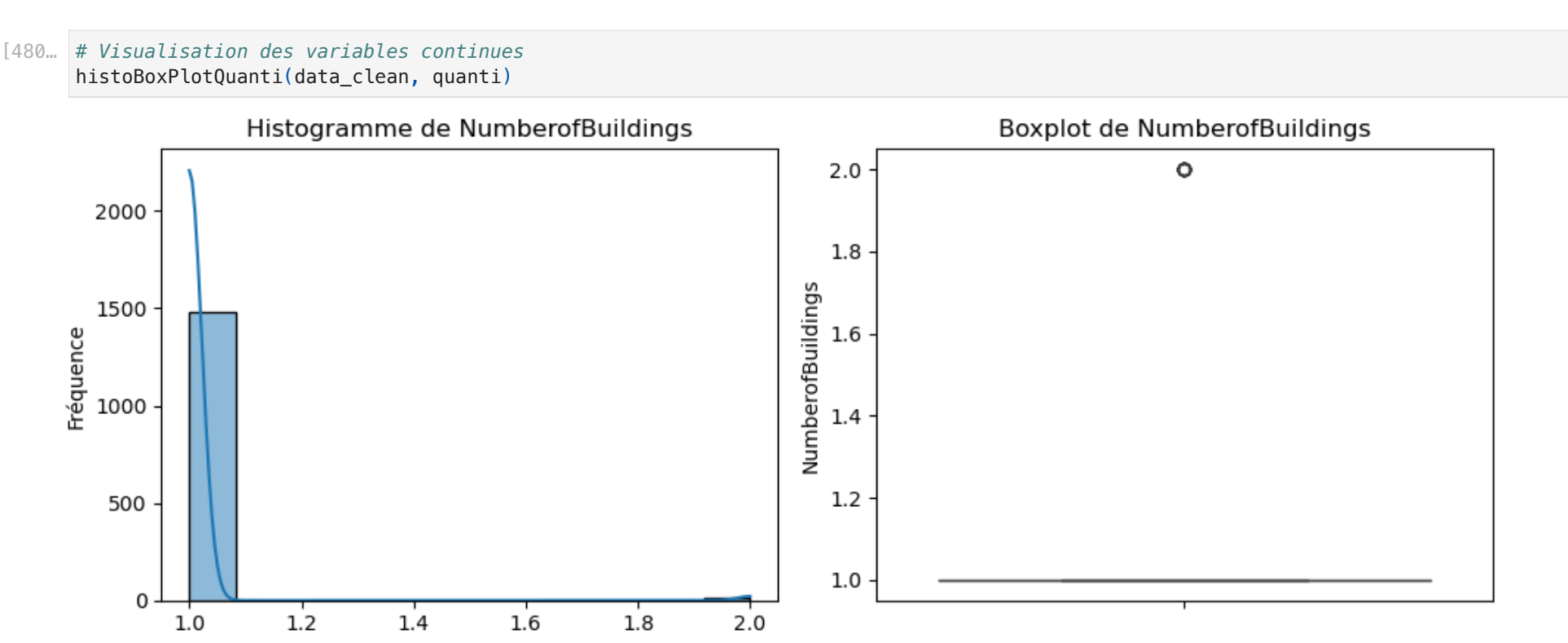
```python
# Visualisation des variables catégorielles
histoBoxPlotQuanti(data_clean, quali)
```

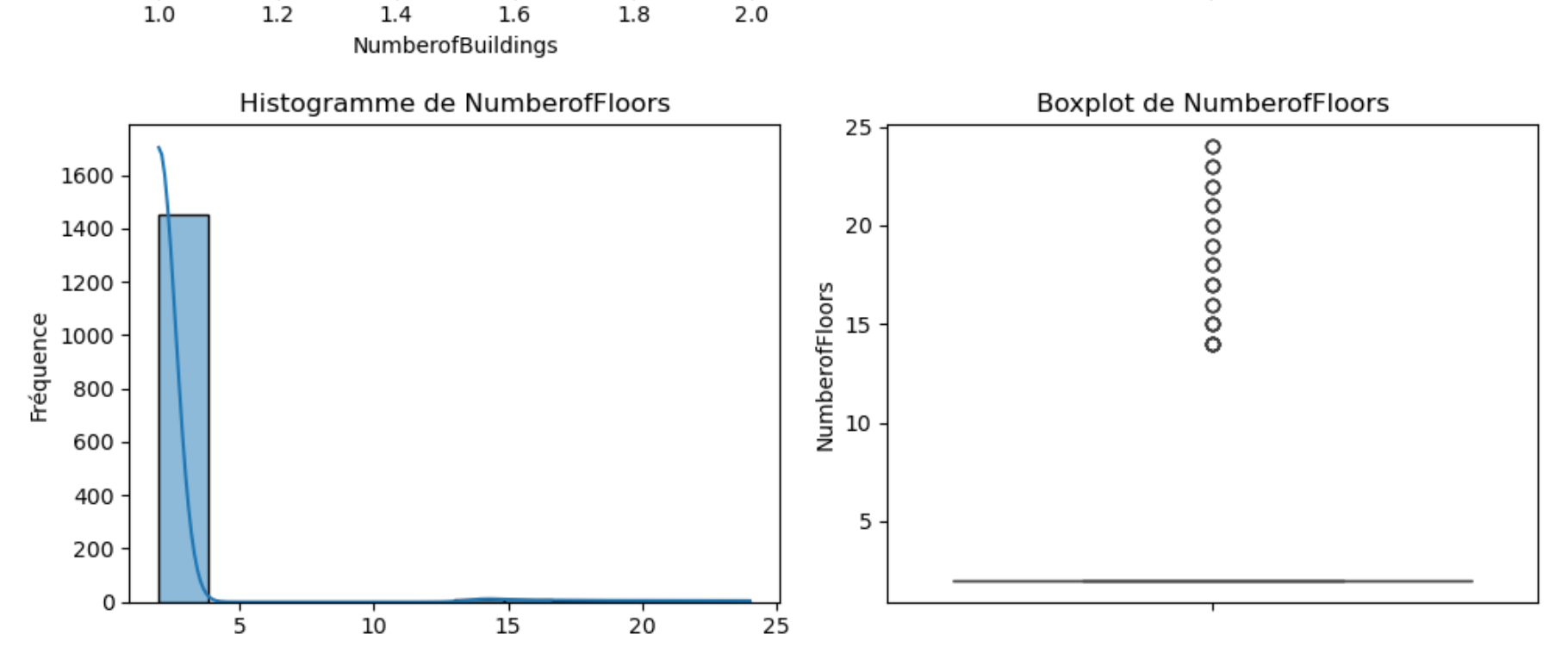


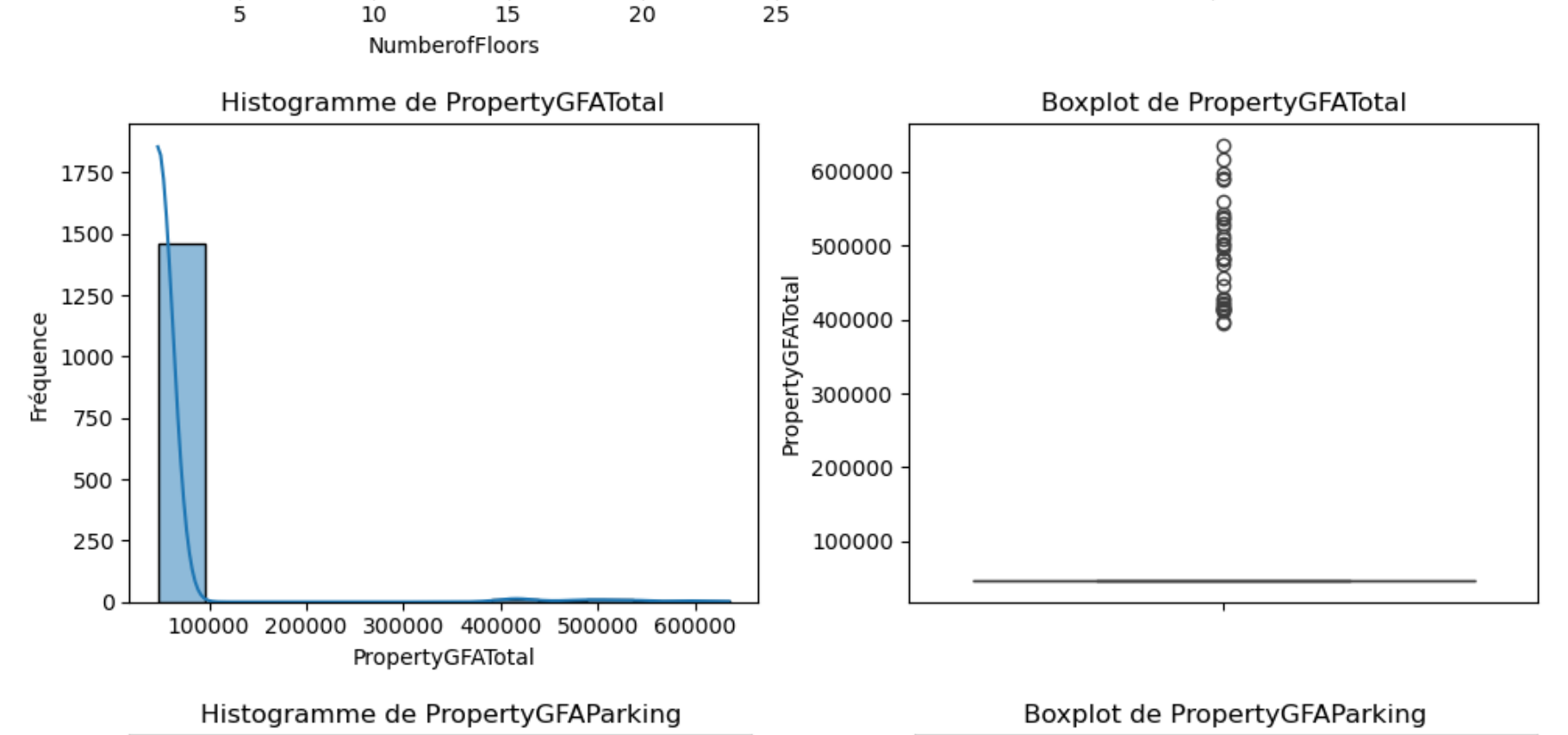


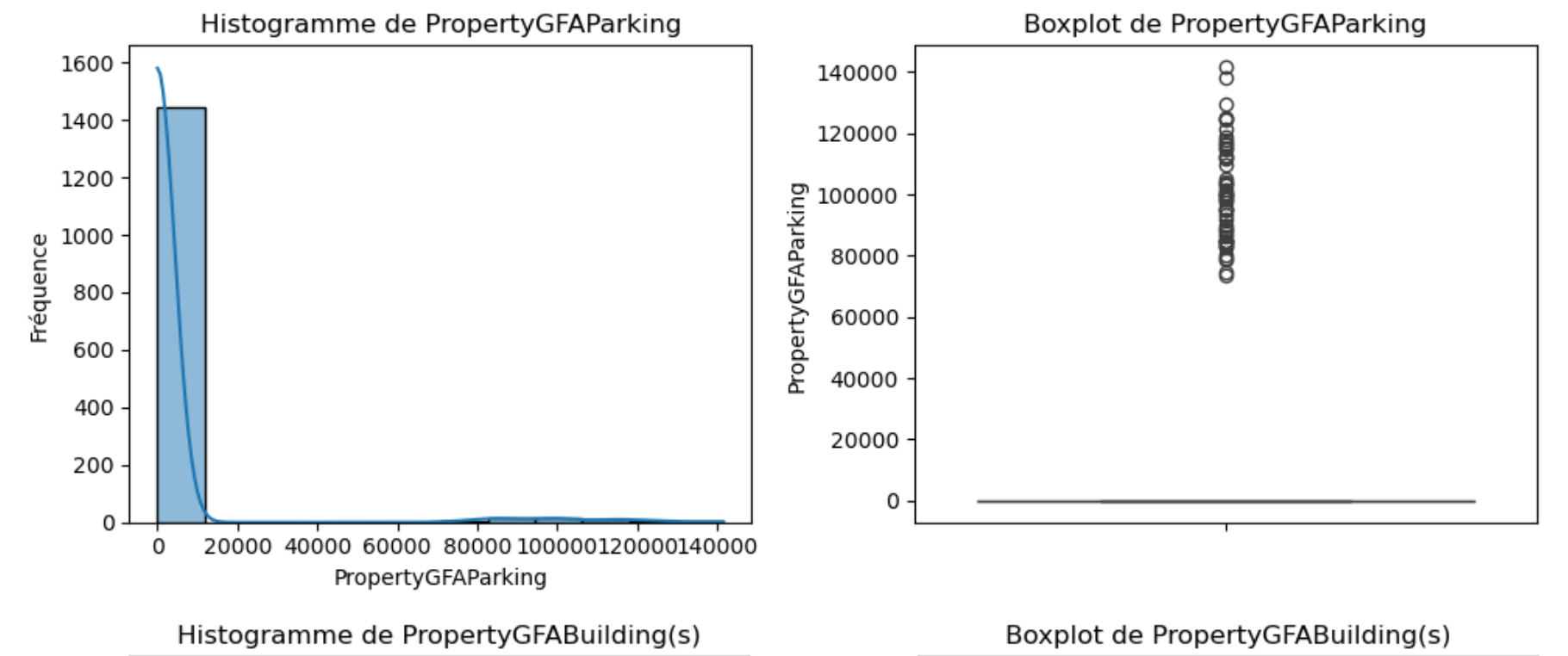
In [480…

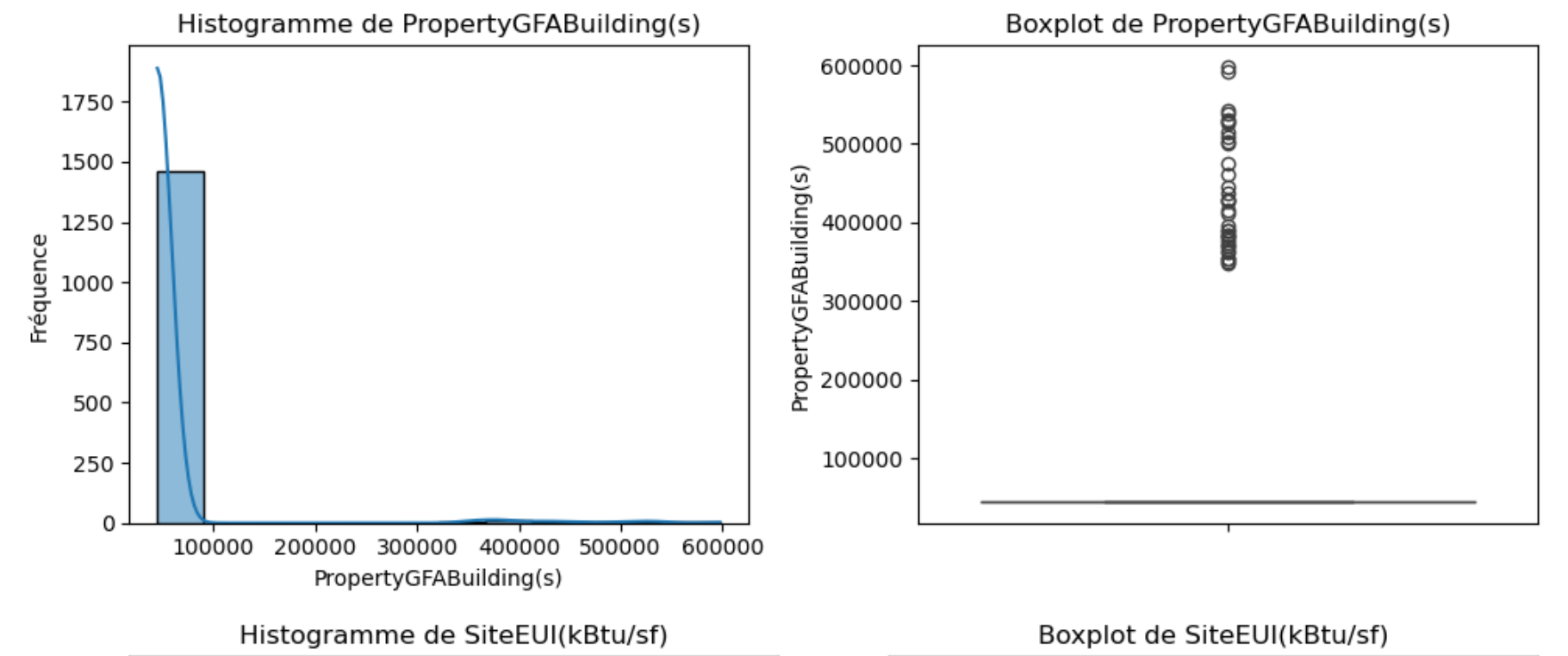
```python
# Visualisation des variables continues
histoBoxPlotQuanti(data_clean, quanti)
```

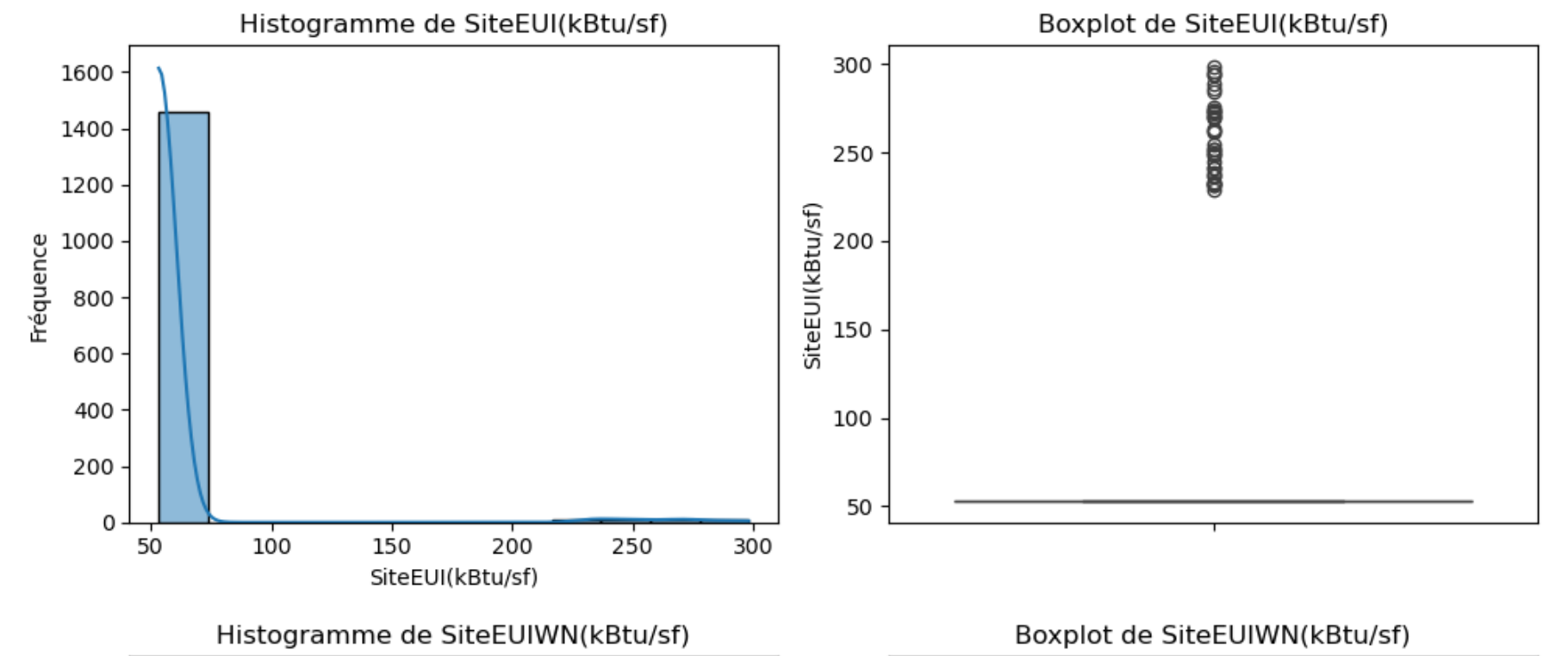


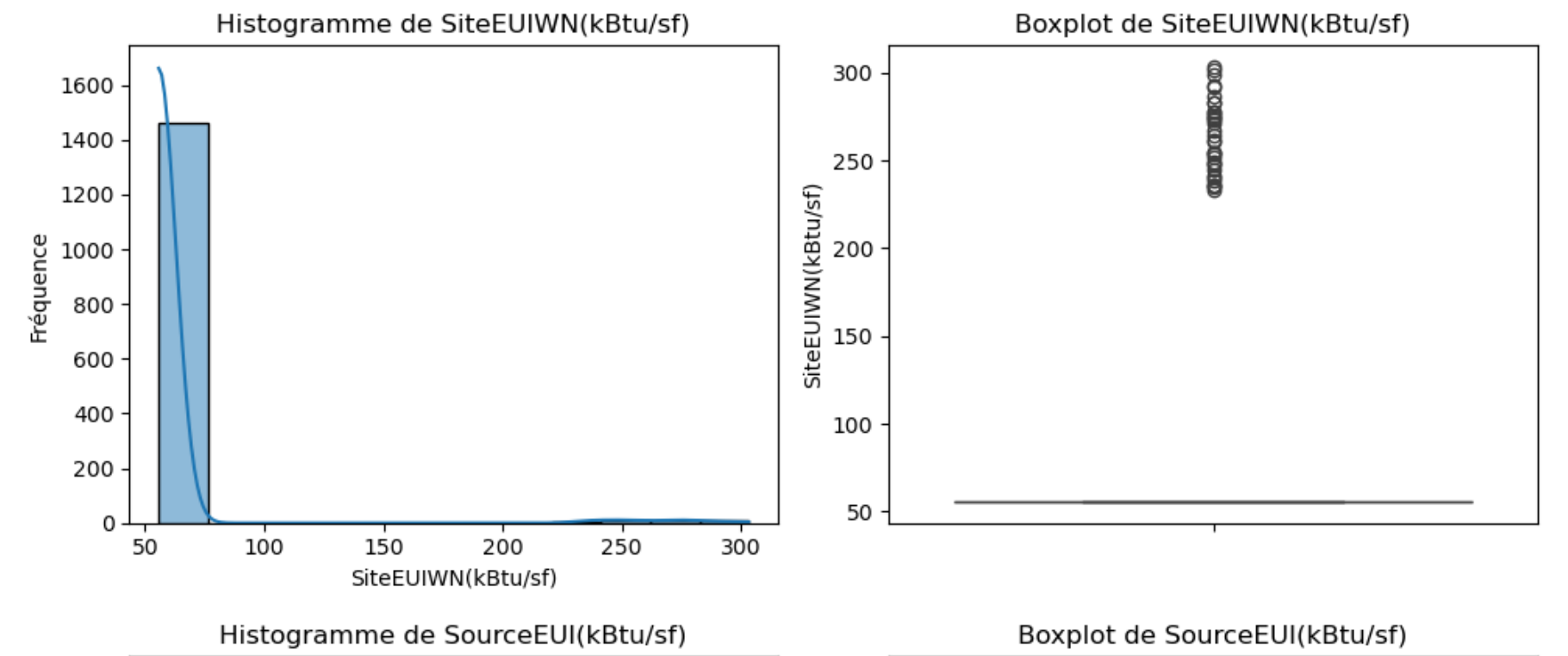


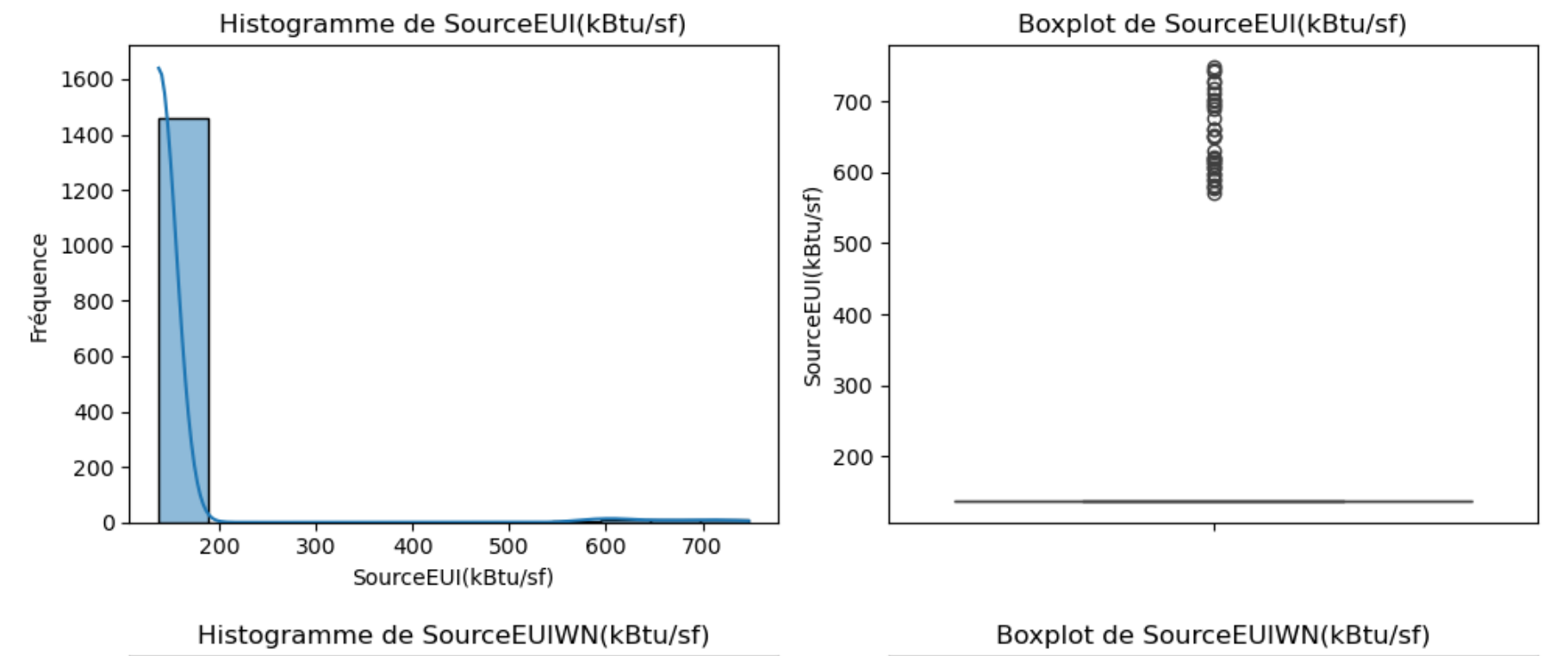


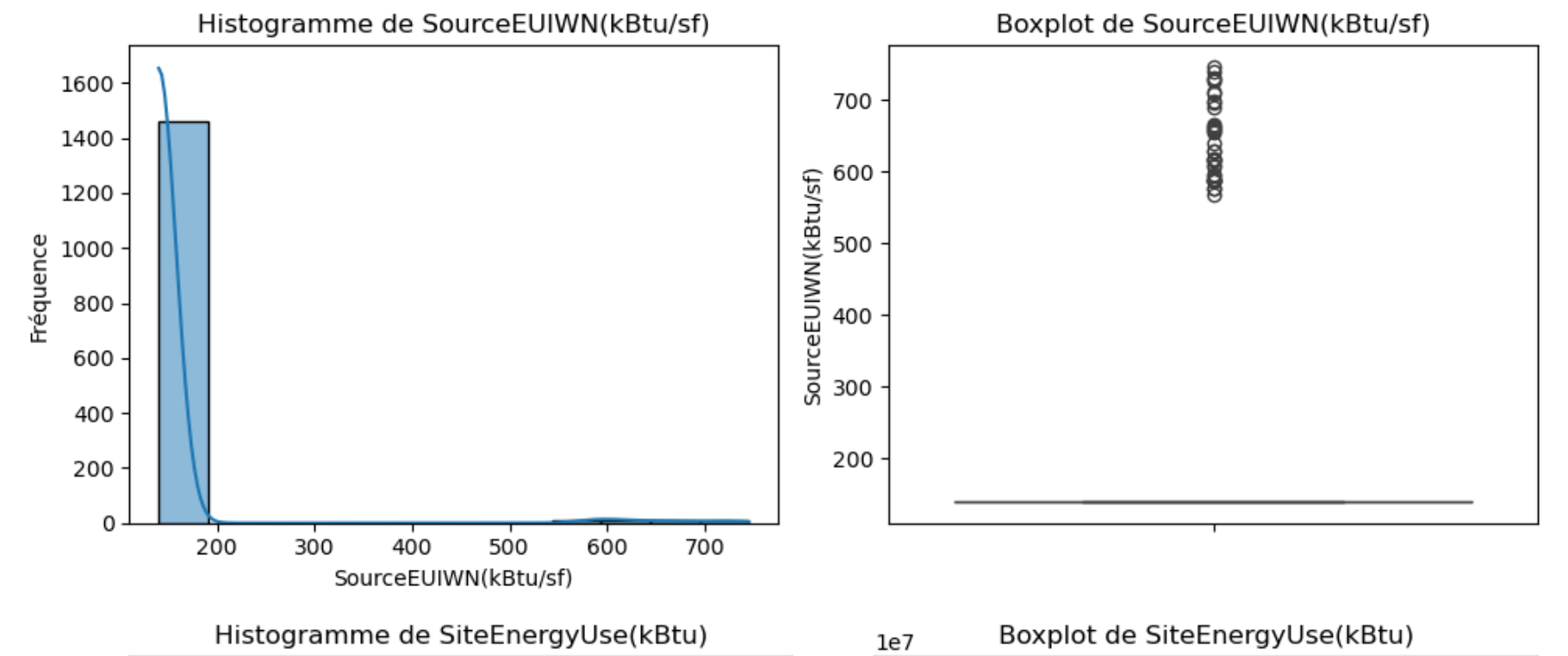


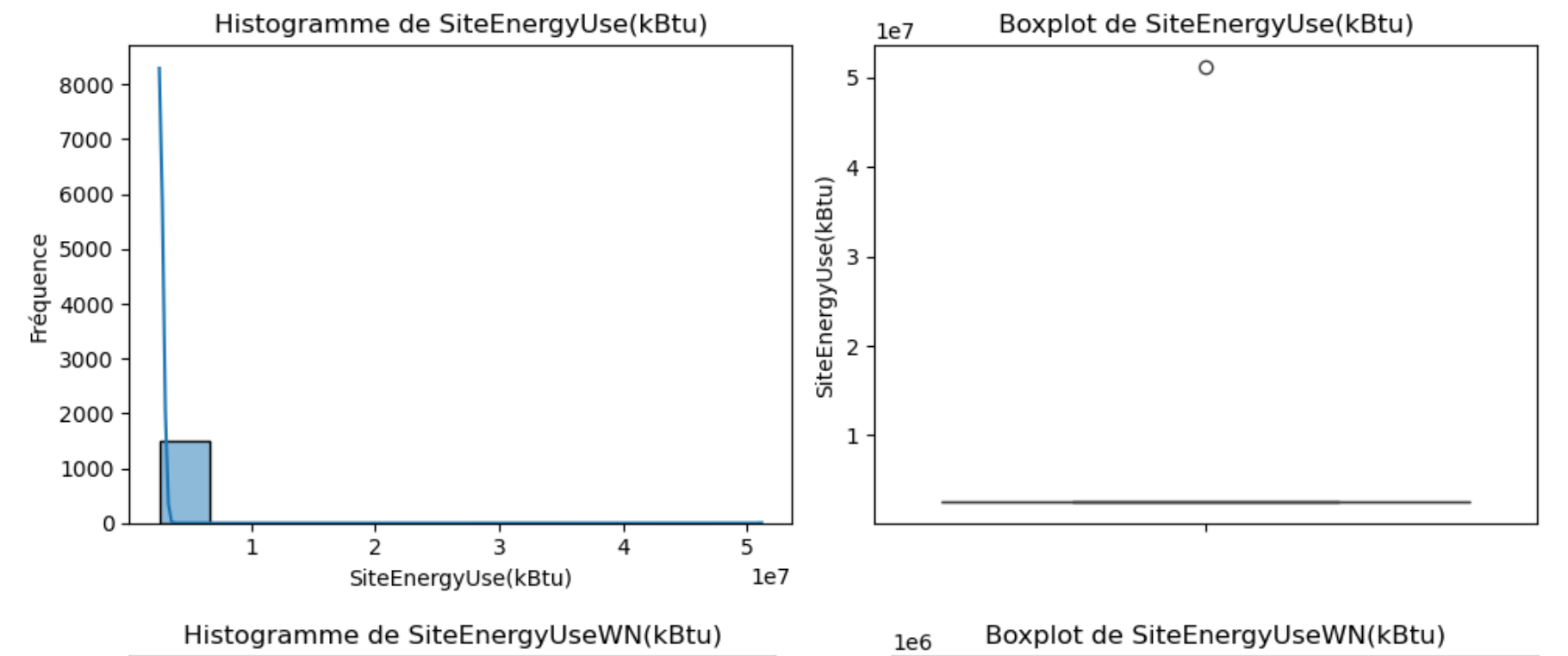


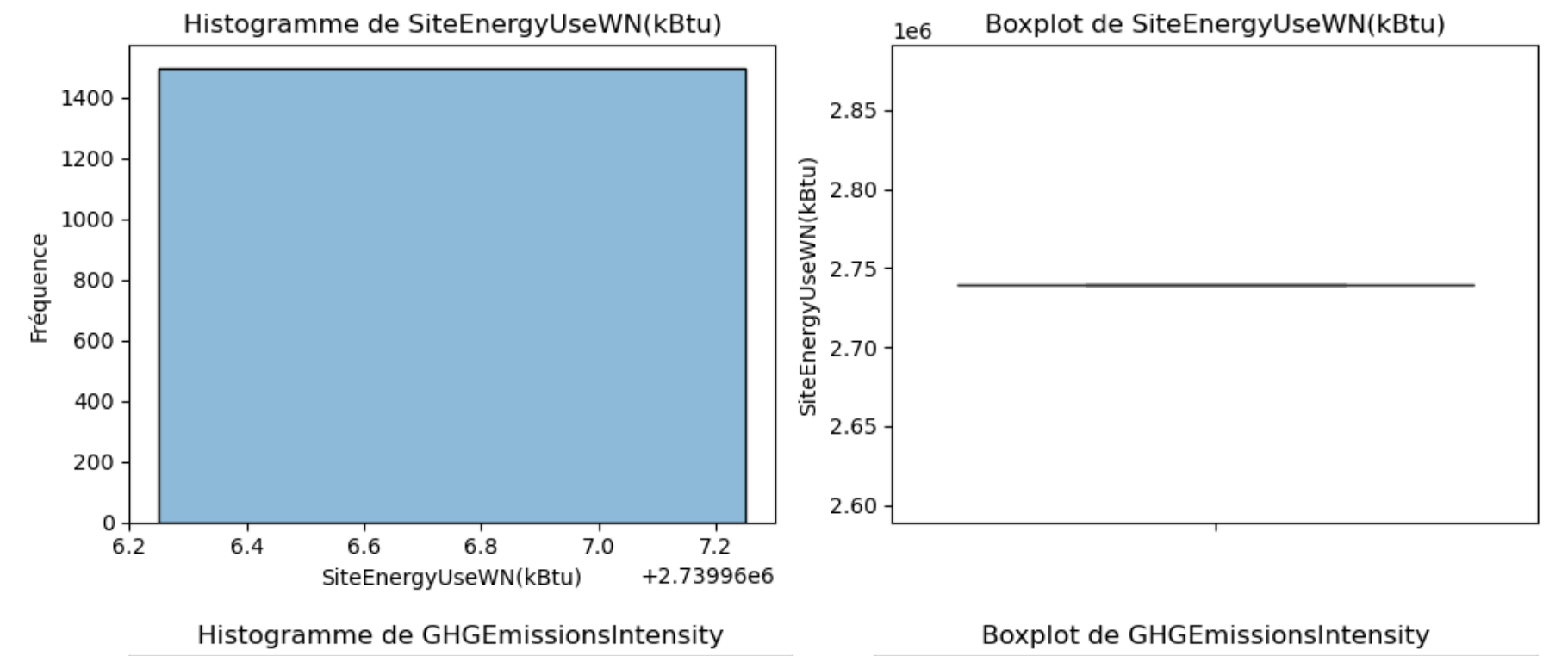


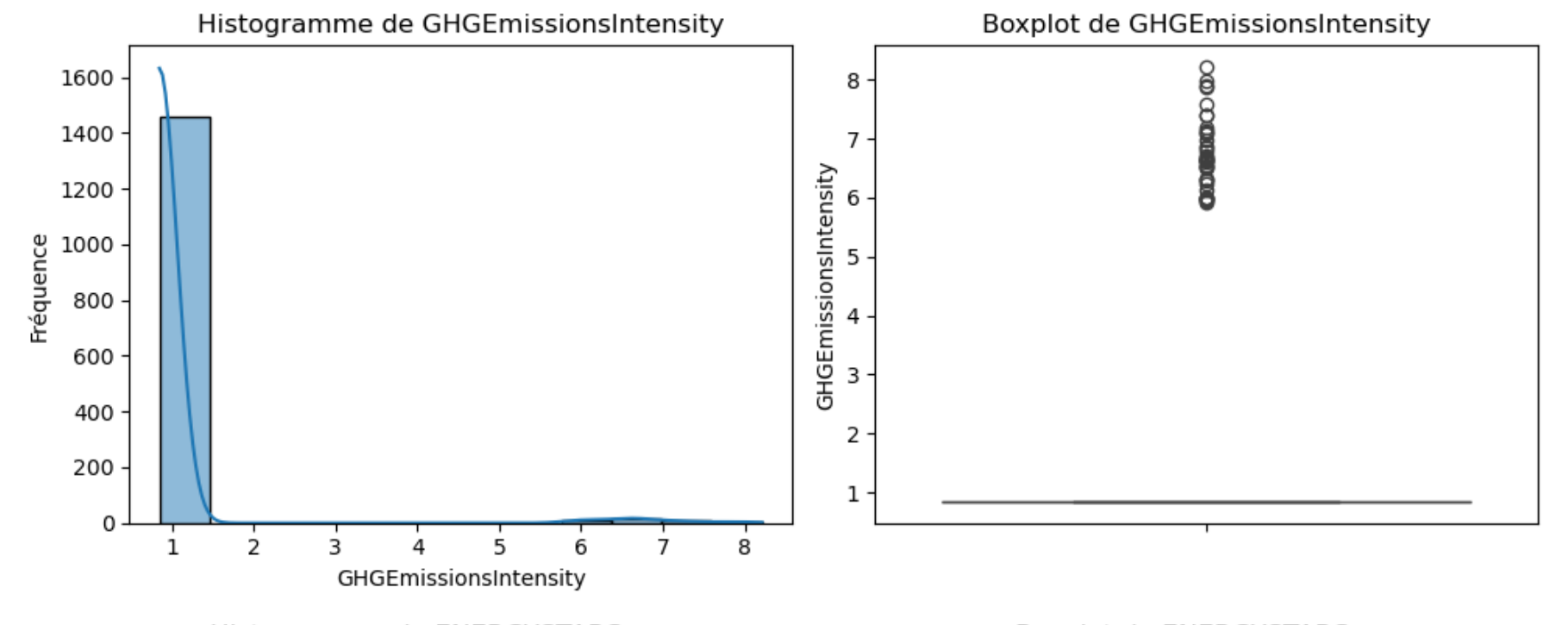


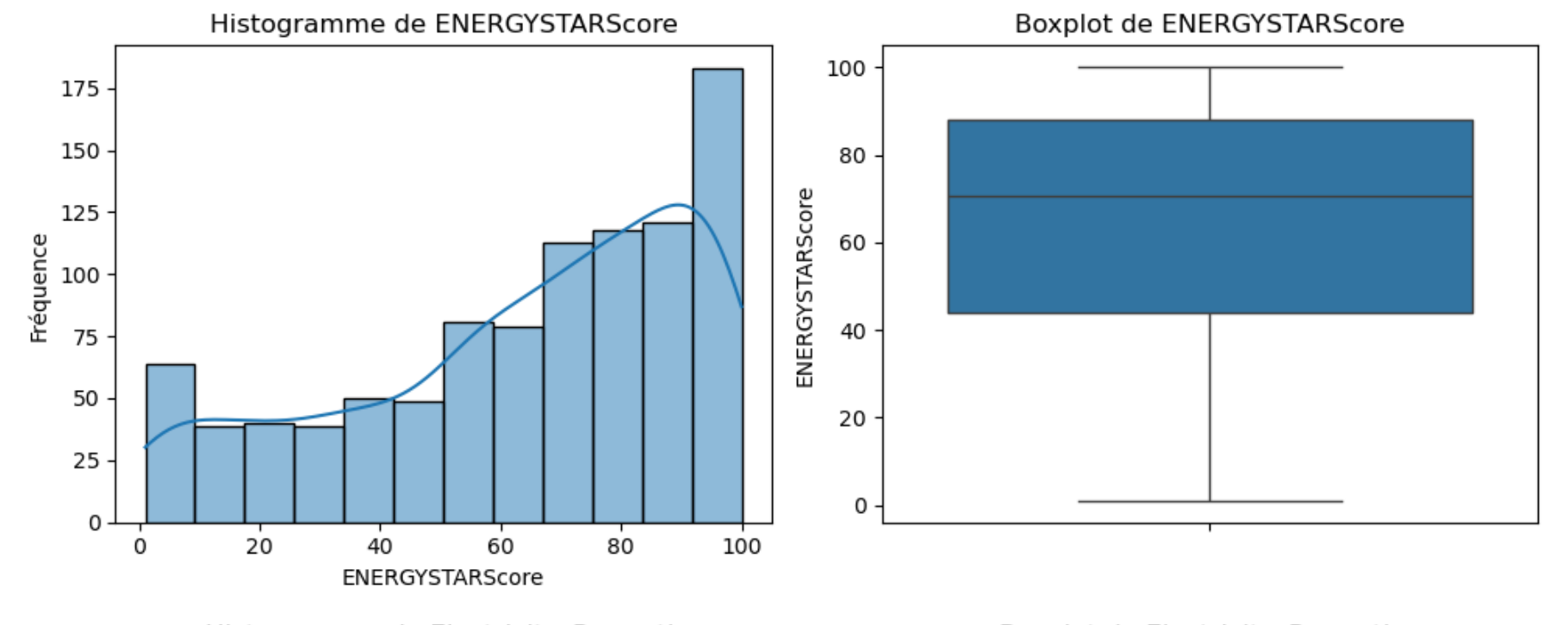


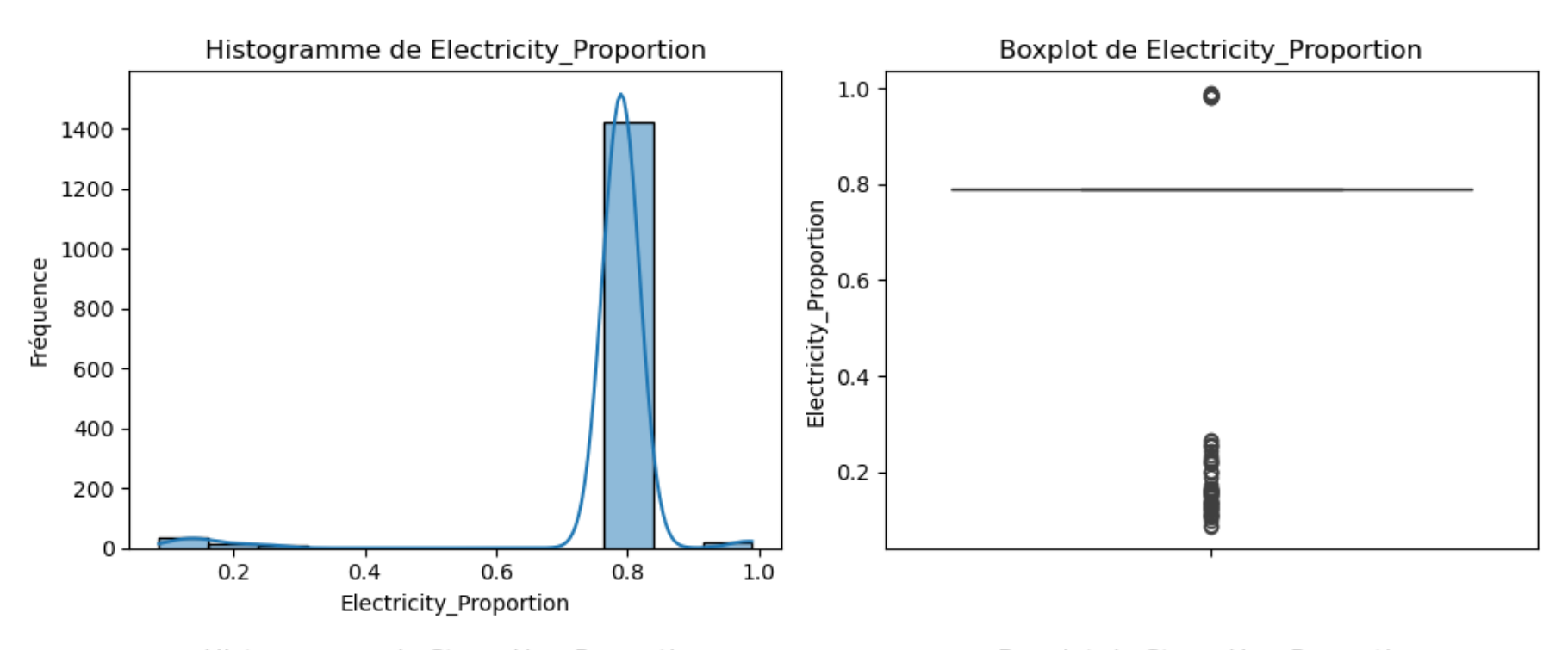


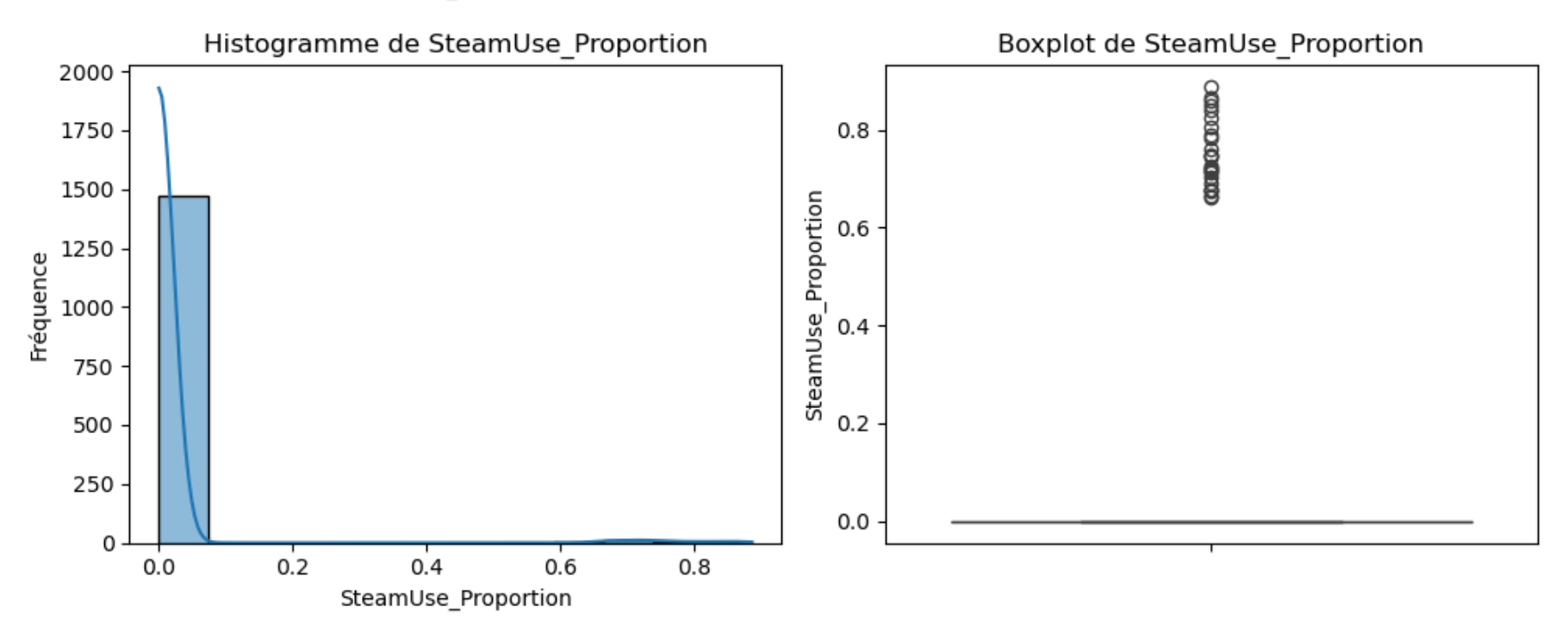


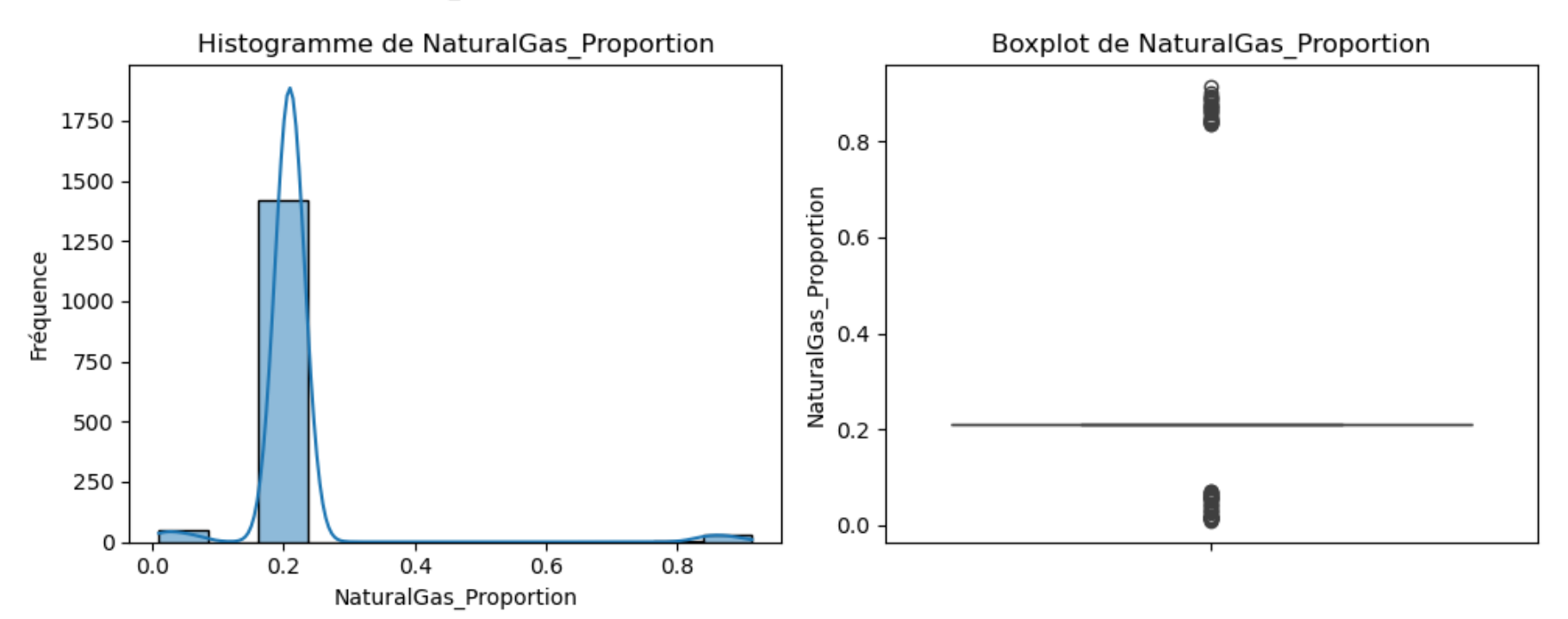












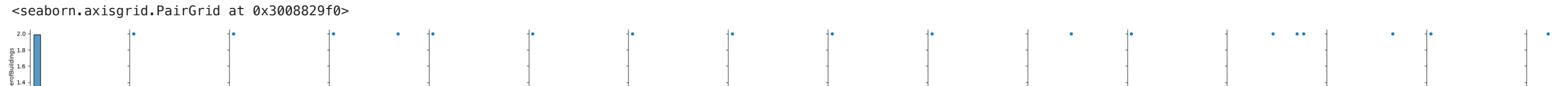
BIVARIEE

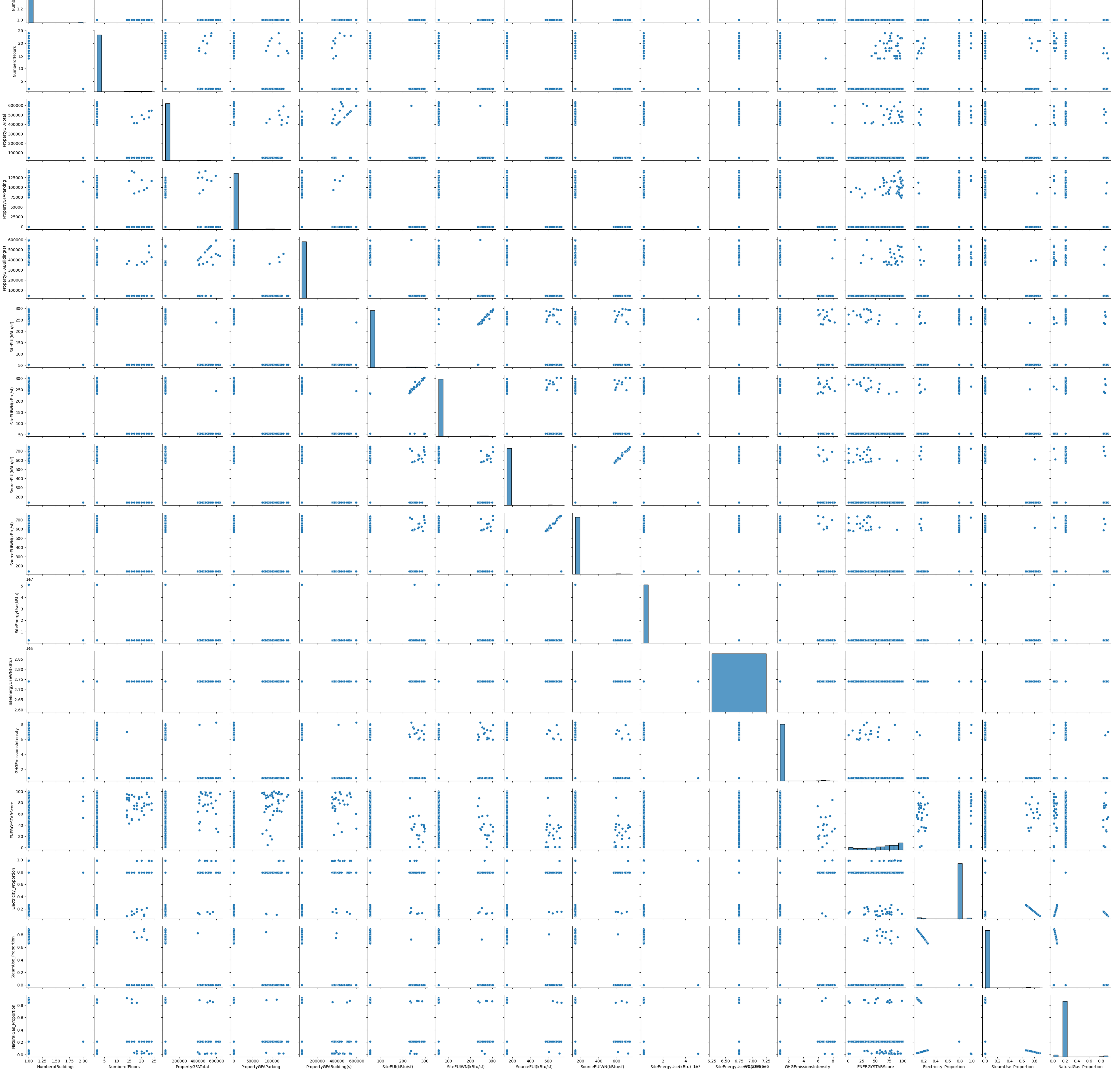
In [482…

```python
sns.pairplot(data_clean[quanti])
```

Out[482…

```
<seaborn.axisgrid.PairGrid at 0x3008829f0>
```



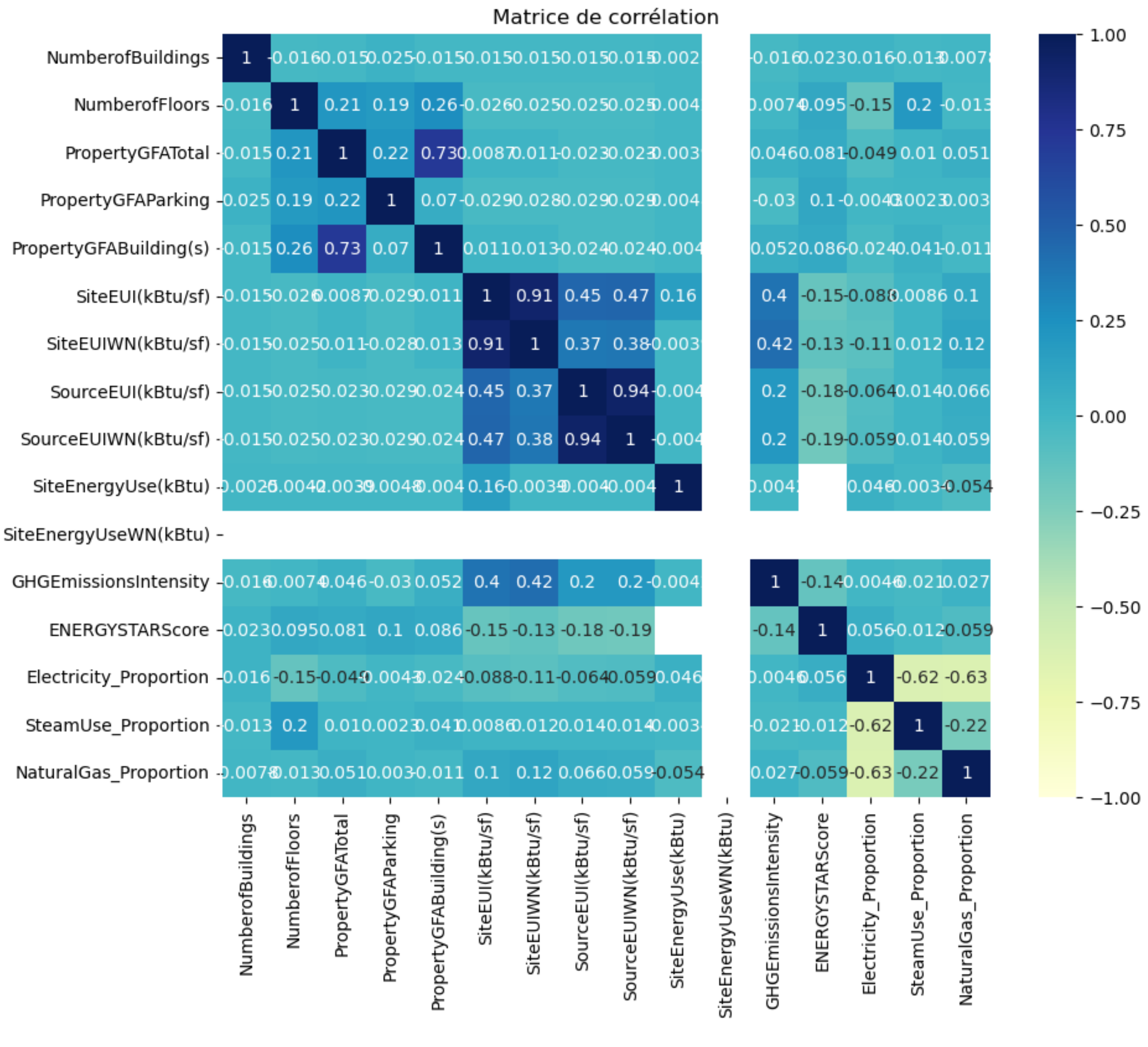


In [484…

```python
def heatmap(data, colonnes, check_normality=False):
    if check_normality:
        for col in colonnes:
            stat, p = st.shapiro(data[col])
            if p <= 0.05:
                print(f"La colonne {col} ne suit pas une distribution normale (p={p:.3f}).")

    # Calculer la matrice de corrélation
    corr_matrix = data[colonnes].corr()

    # Tracer la carte thermique
    plt.figure(figsize=(10, 8))
    sns.heatmap(corr_matrix, annot=True, cmap='YlGnBu', vmin=-1, vmax=1)
    plt.title("Matrice de corrélation")
    plt.show()

heatmap(data_clean, quanti)
```